

Deep Learning

Optimiser l'apprentissage

Théo Mariotte - 06/02/2026

Optimisation de l'apprentissage

Sommaire

4. Réutiliser l'existant

1. Principe du transfer learning
2. Finetuning
3. Foundation models

5. Améliorer la généralisation

1. Weight-decay
2. Drop-out
3. Augmentation de données

6. Fin de l'apprentissage

1. Early stopping
2. Model checkpointing

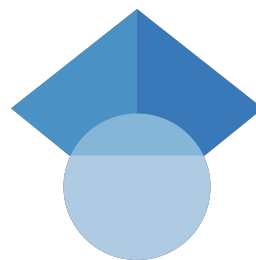
Réutiliser l'existant

Introduction

- Machine learning / deep learning = grande communauté internationale
- La recherche va (très) vite dans le domaine
- Culture de l'open source (dans une certaine mesure)
- **Il faut toujours faire l'état de l'art avant de démarrer un projet !**



<https://arxiv.org/>



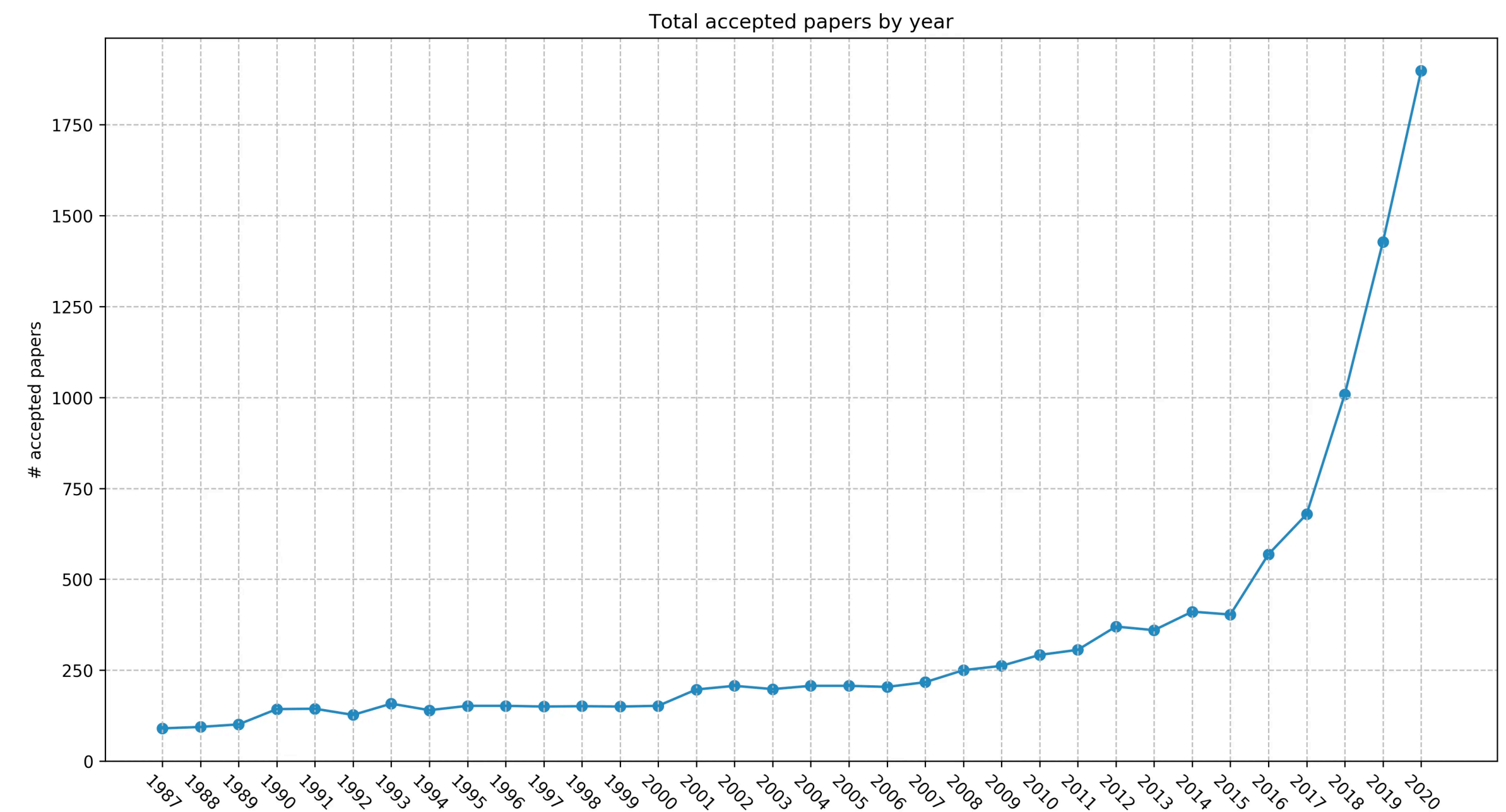
<https://scholar.google.com/>



<https://www.semanticscholar.org/>



<https://www.litmaps.com/>



<https://towardsdatascience.com/neurips-conference-historical-data-analysis-e45f7641d232>

2024: 4 037 papers accepted
(~15000 submissions)

Réutiliser l'existant

Introduction

- Trouver les modèles et les codes

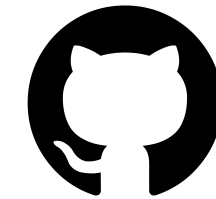


<https://paperswithcode.com/>



Hugging Face

<https://huggingface.co/>



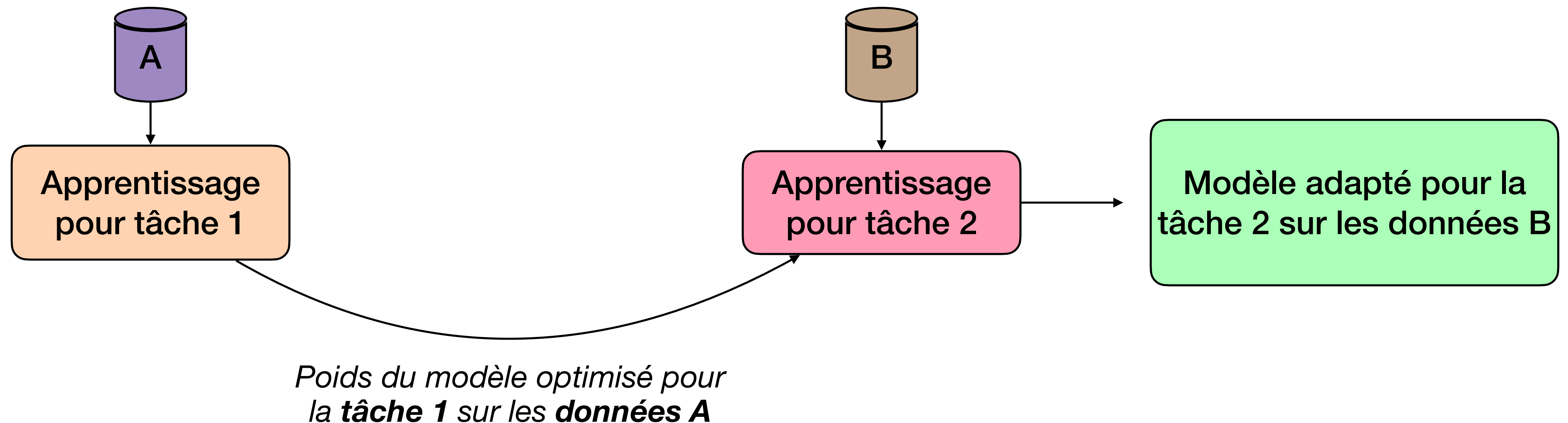
<https://github.com/>

- **ATTENTION :**
 - Vérifiez que vous avez le droit d'utiliser le modèle ET les données utilisées pour l'entraîner
 - Certains jeux de données sont interdits par la législation européenne par exemple
 - Les données d'apprentissage peuvent également être biaisées

Réutiliser l'existant

Transfer-learning

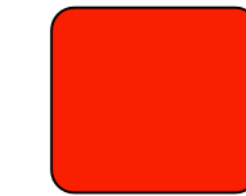
- Le *transfer learning* consiste à utiliser un modèle pré-entraîné pour une nouvelle tâche



- La tâche 2 est généralement proche de la tâche 1

Réutiliser l'existant

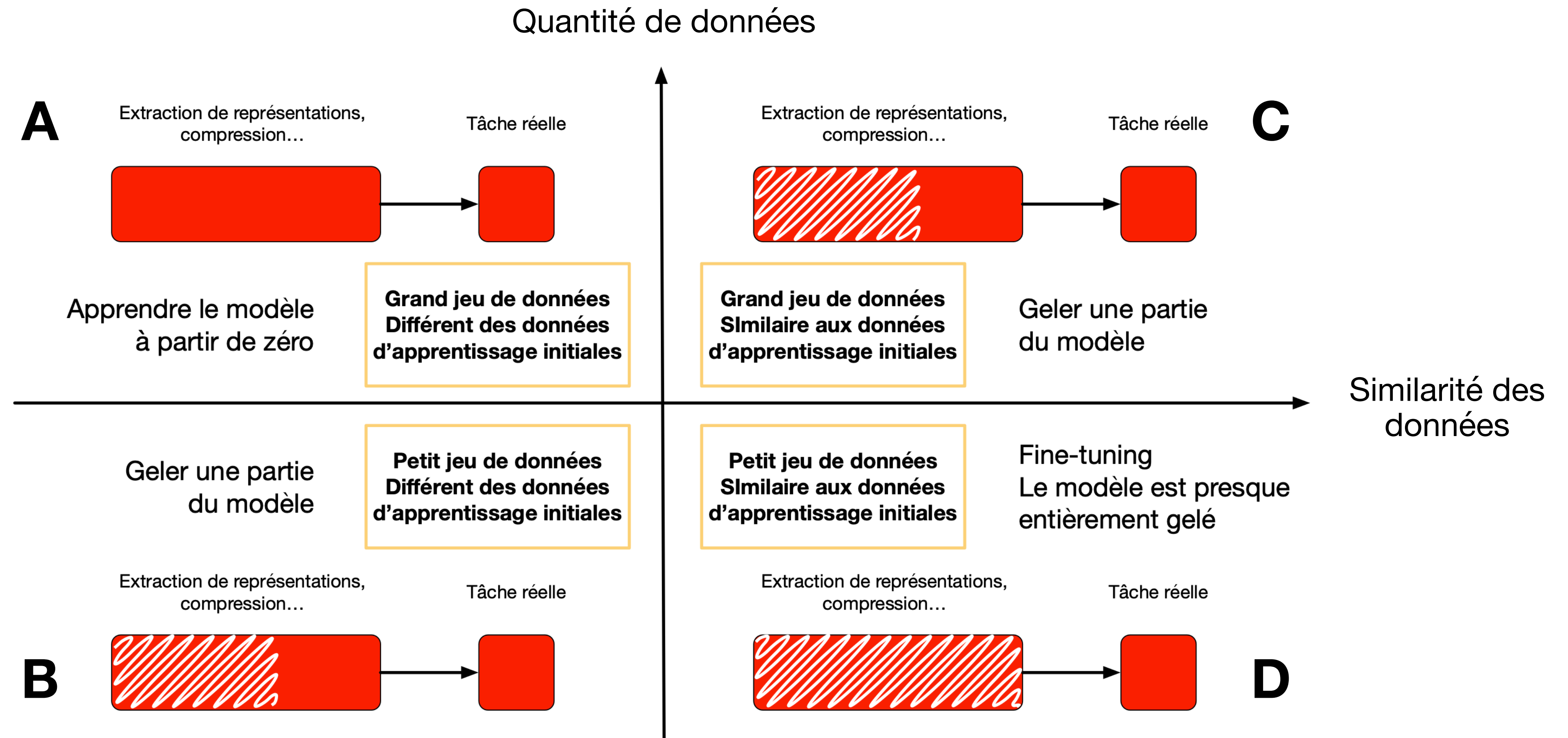
Stratégies de *transfer learning*



Partie du modèle entraîné



Partie du modèle gelée



Réutiliser l'existant

Stratégies de *transfer learning*

- **A** : si on a beaucoup de données très différentes de celles du modèle original :
 - on ré-apprend le modèle complet
 - Possibilité d'initialiser avec les poids du modèle original, mais aucune garantie d'amélioration
- **B** : peu de données différentes de celles du modèle original :
 - Situation la plus délicate
 - On gèle une partie des poids (souvent les premières couches)
- **C** : Beaucoup de données pour une tâche similaire
 - On gèle une partie du modèle et on adapte quelques poids
 - On utilise souvent un learning rate faible pour éviter de dégrader le modèle original
- **D** : peu de données pour une tâche similaire:
 - Il est souvent préférable de geler tout le modèle sauf la sortie
 - Cette opération est couramment appelée *fine tuning*

Réutiliser l'existant

Foundation models

- Il s'agit de modèles entraînés sur de grands jeux de données pouvant s'appliquer à de nombreuses tâches
- Cette notion est apparue à la fin des années 2010 avec l'apparition de l'architecture *Transformer* (M2)
- Ces modèles sont appris par apprentissage autodidacte (*self-supervised* learning)
- Il existe une grande diversité de modèles
 - NLP (TAL) : BERT, GPT, Mistral...
 - Speech (TAP) : Wav2vec, WavLM, XLSR, HuBERT...
 - Multimodaux : DALL-E, Stable Diffusion, CLIP, CLAP...
- Ces systèmes permettent d'obtenir une représentation optimisée des données, et n'ont souvent pas besoin d'être optimisés pour la tâche (ou juste *fine tuned*)
- Cependant, le biais de ces modèles sont mal connus (handle with care)

Optimisation de l'apprentissage

Sommaire

4. Réutiliser l'existant

1. Principe du transfer learning
2. Finetuning
3. Foundation models

5. Améliorer la généralisation

1. Weight-decay
2. Drop-out
3. Augmentation de données

6. Fin de l'apprentissage

1. Early stopping
2. Model checkpointing

Améliorer la généralisation

Risque empirique et erreur de généralisation

- **Hypothèse apprentissage supervisé** : les données d'apprentissage et de test sont échantillonnées indépendamment de distributions identiques
 - Independently from Identical Distributions, I.I.D.
 - $P(X, Y)$ = distribution du train, $Q(X, Y)$ = distribution du test
 - On suppose $P(\cdot) = Q(\cdot)$ (très forte hypothèse, mais très pratique aussi...)
- **Comment savoir si un modèle généralise ?**
 - Apprentissage = minimiser le risque empirique sur le jeu de train

$$J_{\text{emp}}(\theta) := \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i)$$

- Généralisation : minimiser l'erreur de généralisation

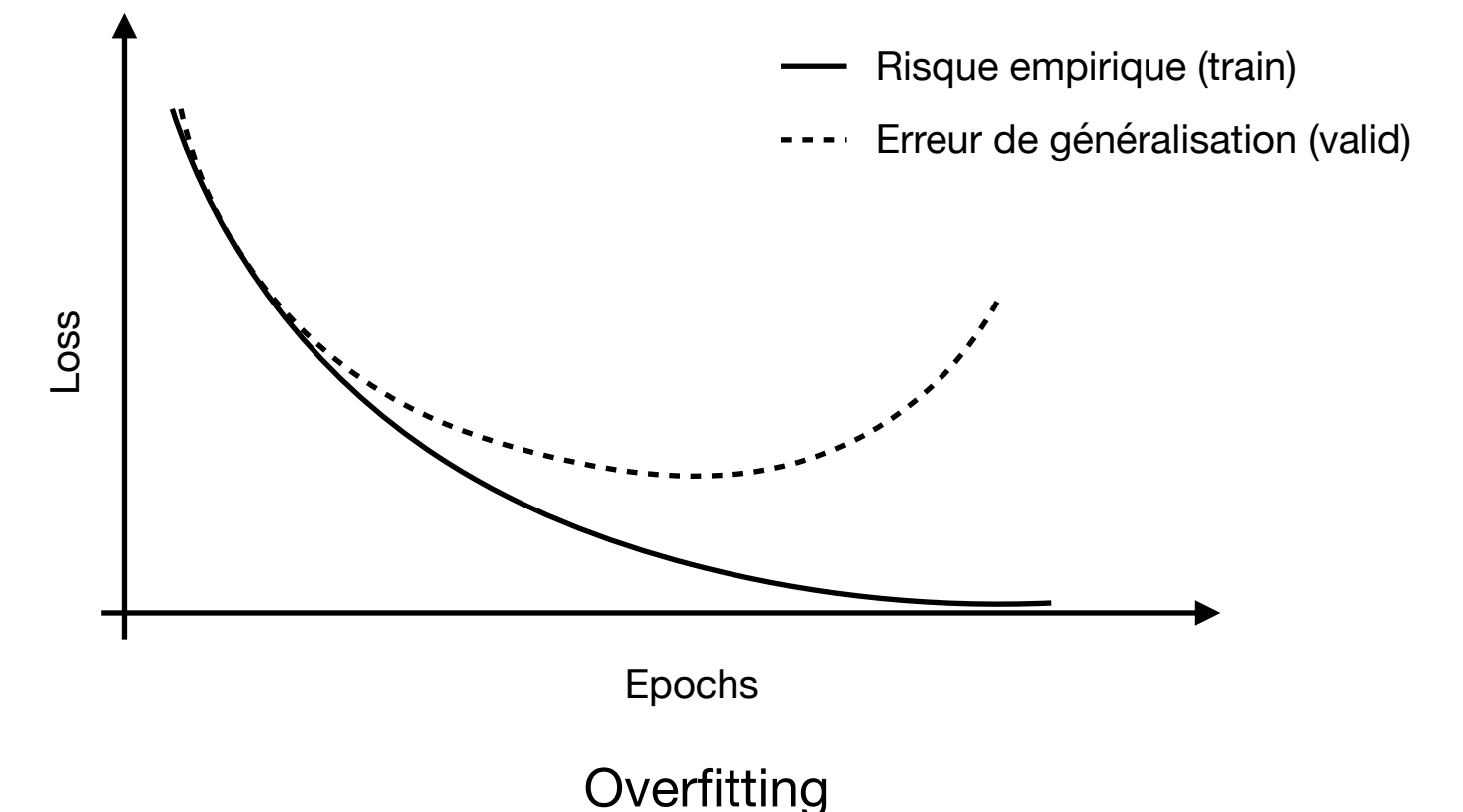
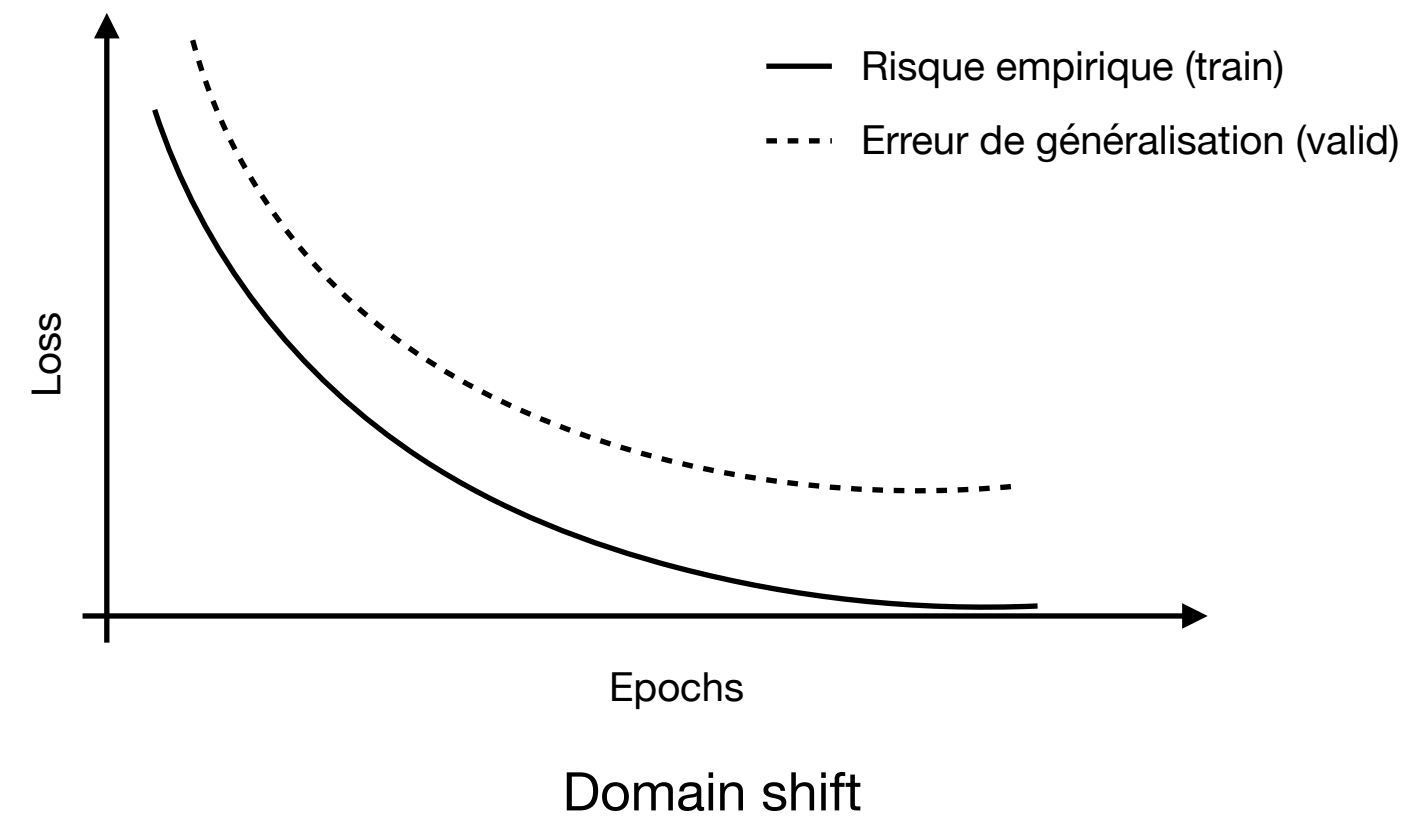
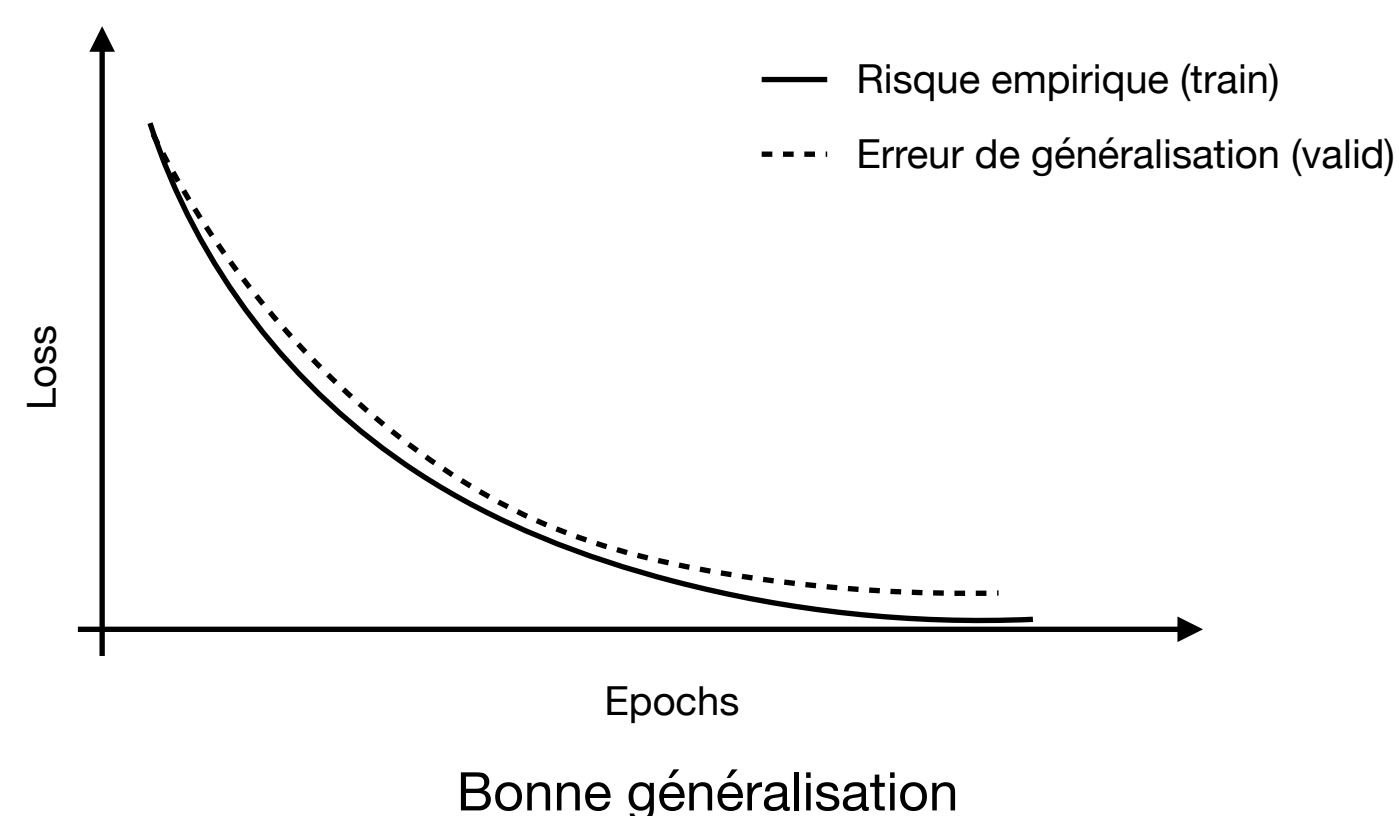
$$\begin{aligned} J(\theta) &= \mathbb{E}_{\mathbf{x}, y \sim P} [\ell(f_{\theta}(\mathbf{x}), y)] \\ &= \iint \ell(f(\mathbf{x}), y) p(\mathbf{x}, y) d\mathbf{x} dy \end{aligned}$$

Comment on calcule ça??

Améliorer la généralisation

Risque empirique et erreur de généralisation

- Comme on a pas accès à $p(\mathbf{x}, y)$, ni à $J(\theta)$, on est voué à utiliser la démarche suivante
 - On minimise $J_{\text{emp}}(\theta)$ sur le jeu d'apprentissage $\mathcal{D}_T$
 - Tout en s'assurant de minimiser $J_{\text{emp}}(\theta)$ sur le jeu de validation $\mathcal{D}_V$ avec $\mathcal{D}_T \cap \mathcal{D}_V = \emptyset$
 - Cela revient à estimer l'erreur de généralisation en appliquant notre modèle *frozen* sur un jeu de données indépendant
 - Toujours garder en tête qu'on travaille avec des **estimateurs statistiques** !



Améliorer la généralisation

Notion de complexité

- Complexité d'un modèle = capacité à modéliser des distributions complexes (ex: multimodales, non-gaussiennes, etc.)
 - Un modèle à faible complexité ne pourra pas apprendre des fonctions complexes mais aura une erreur de généralisation réduite
 - Un modèle très complexe tendra à sur-apprendre, son erreur de généralisation sera élevée
 - Il y a un compromis sur la complexité du modèles et quantité de données
- Exemple : polynomial fit

- $\hat{y} = \sum_{i=1}^d w_i x^i$ avec $\hat{y}$ la prédiction, w_i un poids et x une feature à 1 dimension

- $d = 1 \rightarrow$ faible complexité (régression linéaire) = difficile de modéliser des fonctions complexes, mais bonne généralisation (underfitting)
- $d = 5 \rightarrow$ polynôme de degré 5 = permet de modéliser des fonctions complexes, mais bien plus sujet au sur-apprentissage (overfitting)
- Est-ce que ça vous rappelle quelque chose ? (Indice, c'était au semestre précédent)

Améliorer la généralisation

Comment faire, pour améliorer la généralisation ?

- Il est très courant d'être insatisfait du comportement de l'erreur de validation 😭
 - Soit elle est trop élevée par rapport à l'erreur d'apprentissage
 - Soit elle augmente au bout de quelques epochs
 - Soit elle est très instable
- **Plusieurs leviers s'offrent à nous ! Quelles sont vos intuitions ?**

Améliorer la généralisation

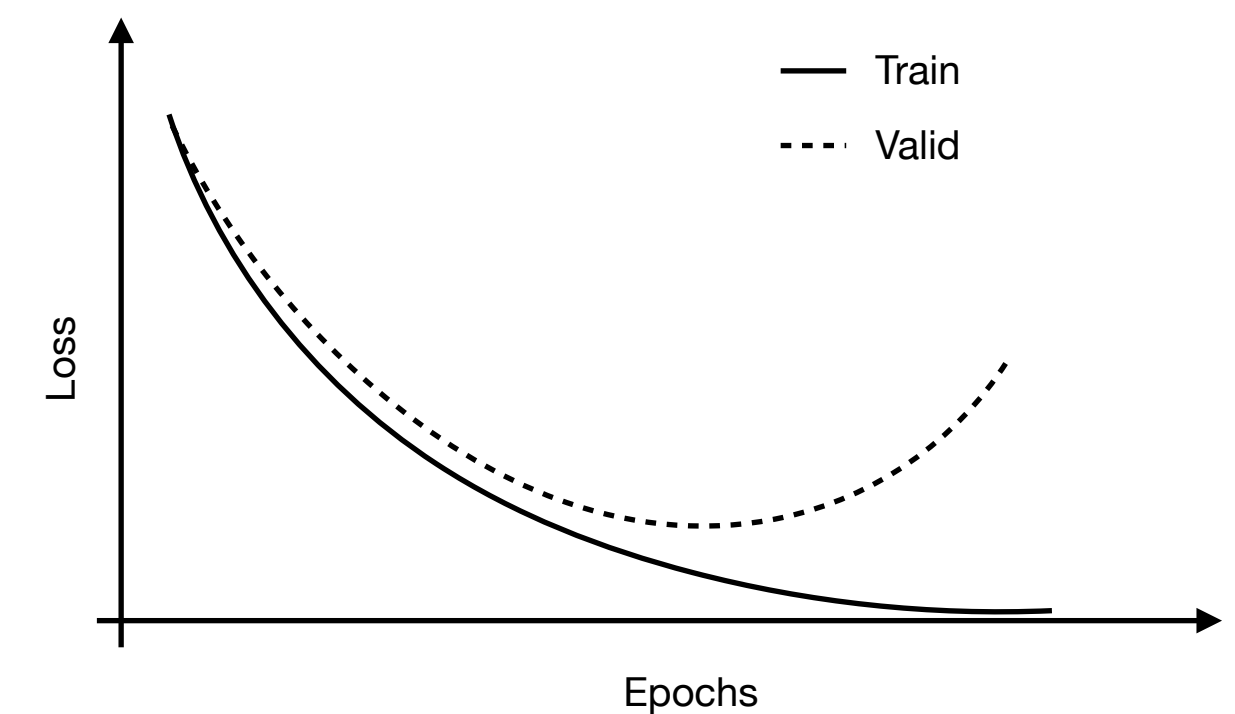
Comment faire, pour améliorer la généralisation ?

- Il est très courant d'être insatisfait du comportement de l'erreur de validation 😭
 - Soit elle est trop élevée par rapport à l'erreur d'apprentissage
 - Soit elle augmente au bout de quelques epochs
 - Soit elle est très instable
- **Plusieurs leviers s'offrent à nous ! Quelles sont vos intuitions ?**
 - Vérifier son jeu de validation (est-ce que tous les labels sont représentés ? Quelle distribution ? Etc.)
 - On peut réduire la complexité du modèle, ou au moins la contrôler pendant l'apprentissage (régularisation)
 - On peut rendre la minimisation de l'erreur de train plus difficile (drop-out)
 - On peut modifier les données pour complexifier l'apprentissage de la fonction $f_{\theta} : \mathbf{x} \mapsto y$

Améliorer la généralisation

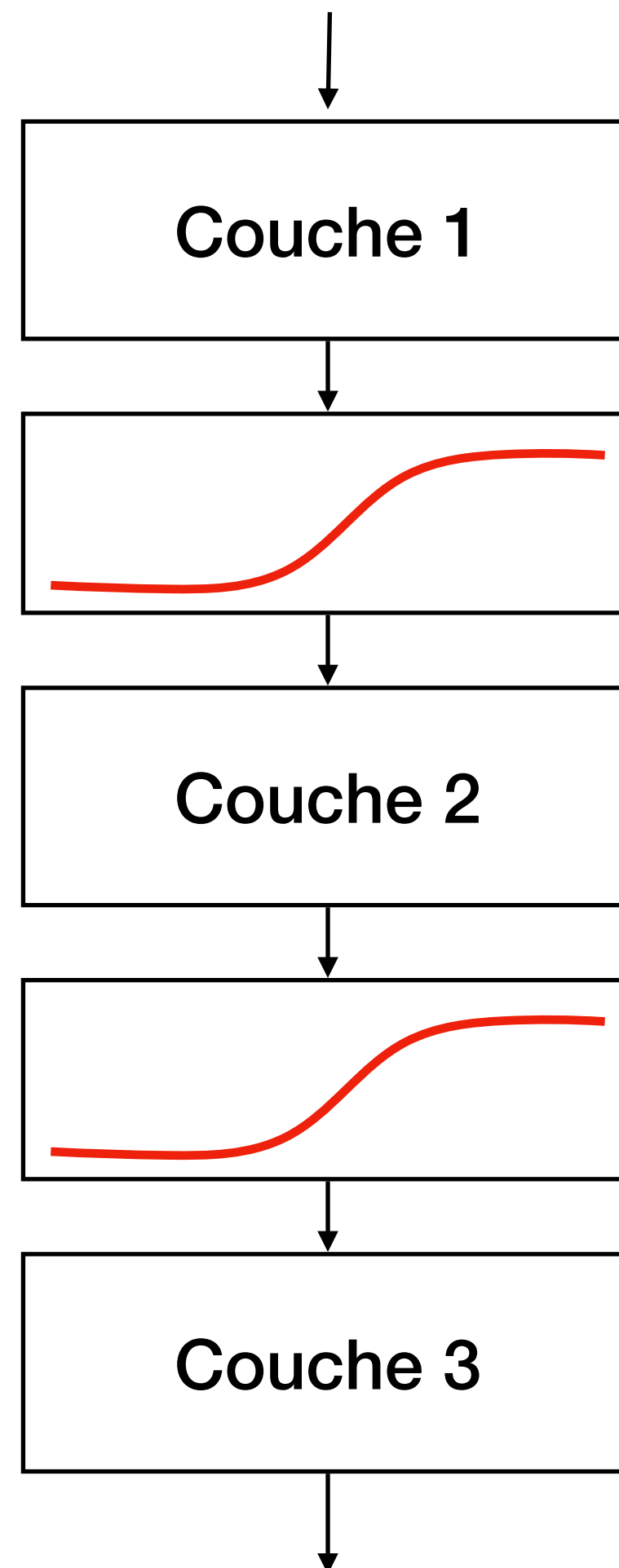
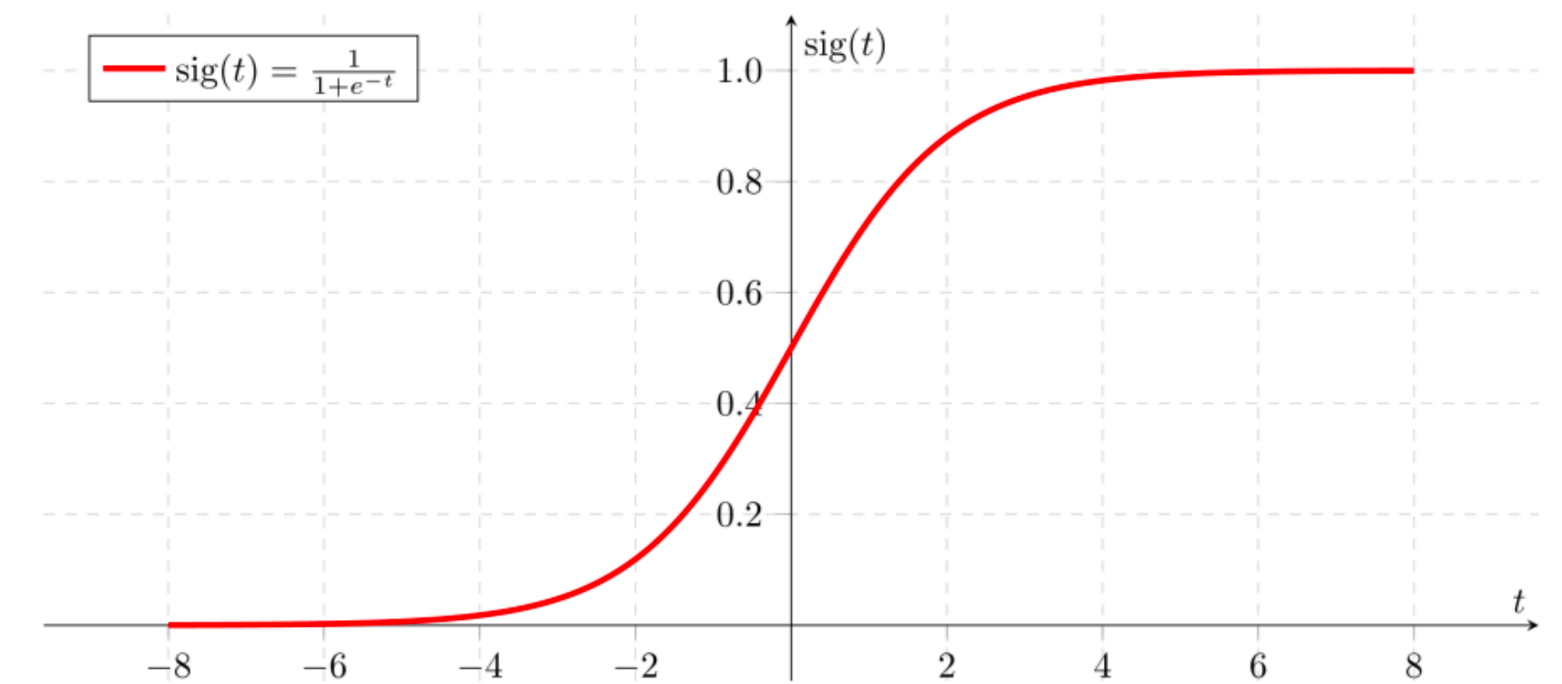
Introduction à la régularisation

- Après entraînement, **les performances en production peuvent fortement différer de l'apprentissage**
- Le modèle est sur-appris (overfitting)
 - Le risque empirique est faible mais l'erreur de généralisation est élevée
- Il a appris « par coeur » les données d'apprentissage
- **Quelques axes pour éviter ça :**
 - Moyenner plusieurs modèles (bagging, boosting)
 - Augmenter la quantité de données d'apprentissage (ça coûte cher)
 - Réduire la complexité du modèle
 - Rendre l'apprentissage plus difficile (régularisation, augmentation de données)



Améliorer la généralisation

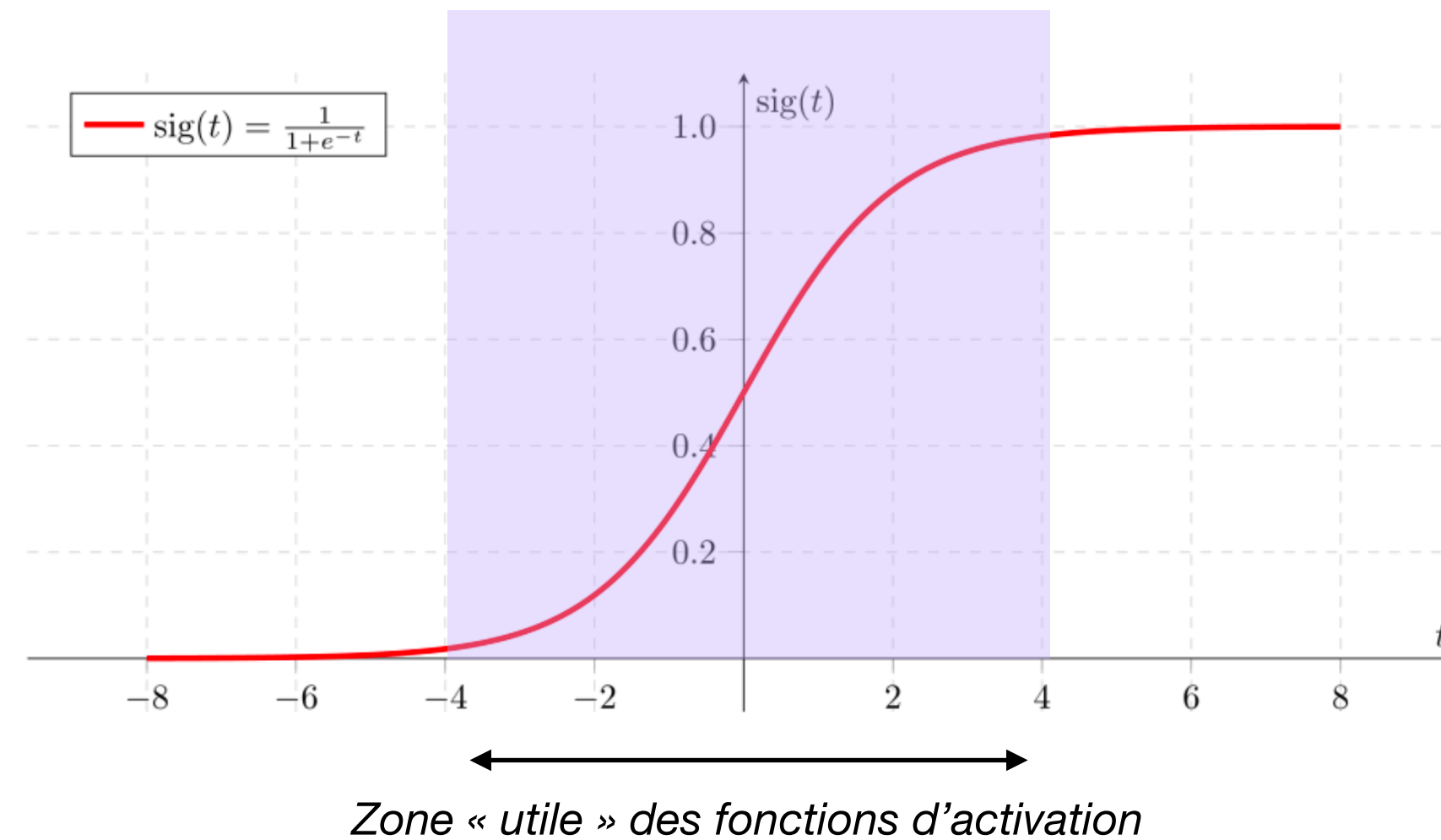
Weight decay



- Fonctions d'activation non-linéaire = puissance des réseaux de neurones
- **Exemple :**
 - couche linéaire + activation : $\mathbf{x}_l = \sigma(\mathbf{W}_l \mathbf{x}_{l-1} + \mathbf{b}_l)$
 - Sigmoid : $\sigma(x) = \frac{1}{1 + e^{-x}}$
 - Si x évolue autour de 0, $\sigma(x)$ a un comportement linéaire
 - $\lim_{x \rightarrow +\infty} \sigma(x) = 1, \lim_{x \rightarrow -\infty} \sigma(x) = 0$ (saturation)
- Si les poids $\mathbf{W}$ deviennent grand $\Rightarrow \mathbf{W}\mathbf{x}$ risque d'être grand $\Rightarrow$ les fonctions d'activations vont saturer $\Rightarrow$ le modèle sur-apprend

Améliorer la généralisation

Weight decay

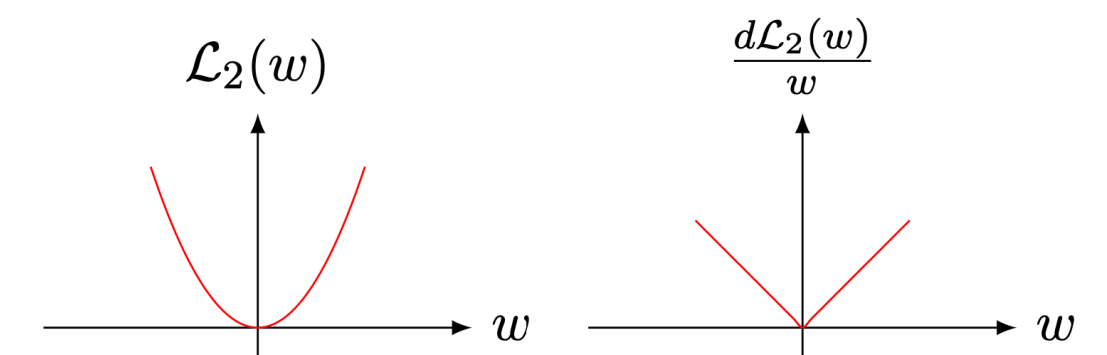


- Pendant l'apprentissage, il faut éviter que les **poids** prennent des **valeurs trop élevées**
- **Weight decay** = imposer une pénalité aux poids pour réduire leur valeur
- Cela revient à ajouter un terme à la fonction de coût prenant en compte la norme des paramètres
- **Le problème de minimisation devient un problème d'optimisation sous contraintes**
- Deux régularisations :
 - Norme L2 $\Rightarrow$ somme des carrés des paramètres (Ridge)
 - Norme L1 $\Rightarrow$ Somme des valeurs absolues des paramètres (LASSO)

$$\mathcal{L} = \ell(\mathbf{y}, \hat{\mathbf{y}}) + \alpha \|\theta\|_p$$

Améliorer la généralisation

Régularisation L2 - Ridge



- La régression L2 s'écrit :

$$\mathcal{L} = \ell(\hat{y}, y) + \frac{\lambda}{2} \sum \theta^2$$

$$\partial_{\theta} \left[\frac{\lambda}{2} \sum \theta^2 \right] = \lambda \theta$$

- Mise à jour des paramètres :

$$\begin{aligned} \theta_{t+1} &= \theta_t - \eta \frac{\partial \mathcal{L}}{\partial \theta} \\ &= \theta_t - \eta \left[\frac{\partial \ell}{\partial \theta} + \lambda \theta \right] \\ &= \theta_t (1 - \eta \lambda) - \eta \frac{\partial \ell}{\partial \theta} \end{aligned}$$

Réduit la mise à jour d'un facteur
dépendant de la valeur du paramètre et
du *learning rate*

- Le weight decay L2 fait tendre les poids vers 0, et prend moins d'importance lorsque quand les poids diminuent

Améliorer la généralisation

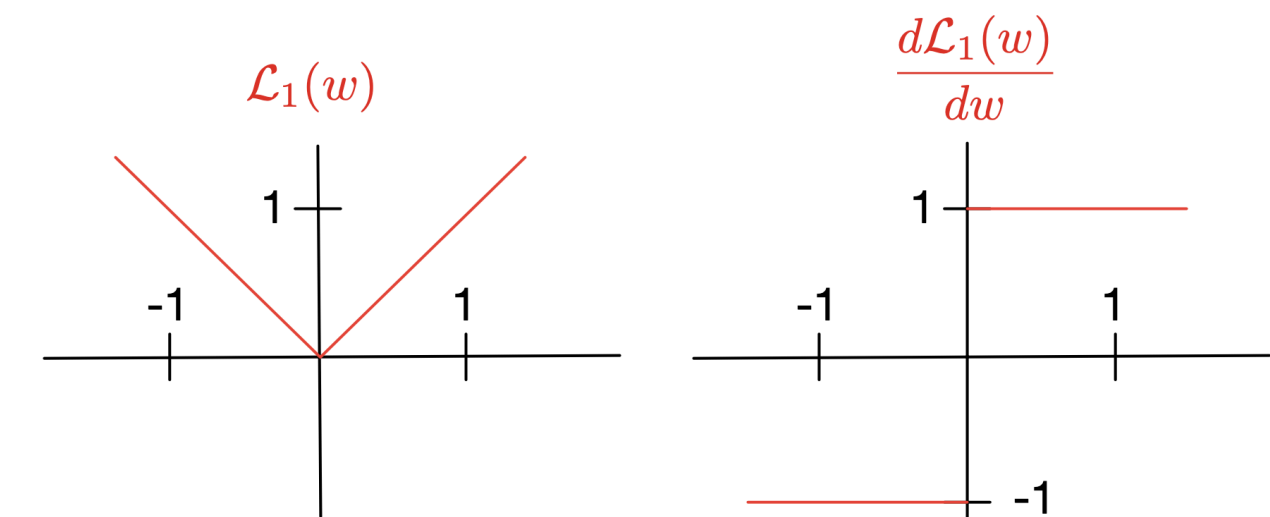
Régularisation L1 - LASSO

- La régression LASSO s'écrit :

$$\mathcal{L} = \ell(\hat{y}, y) + \lambda \sum |\theta|$$

- Mise à jour des paramètres :

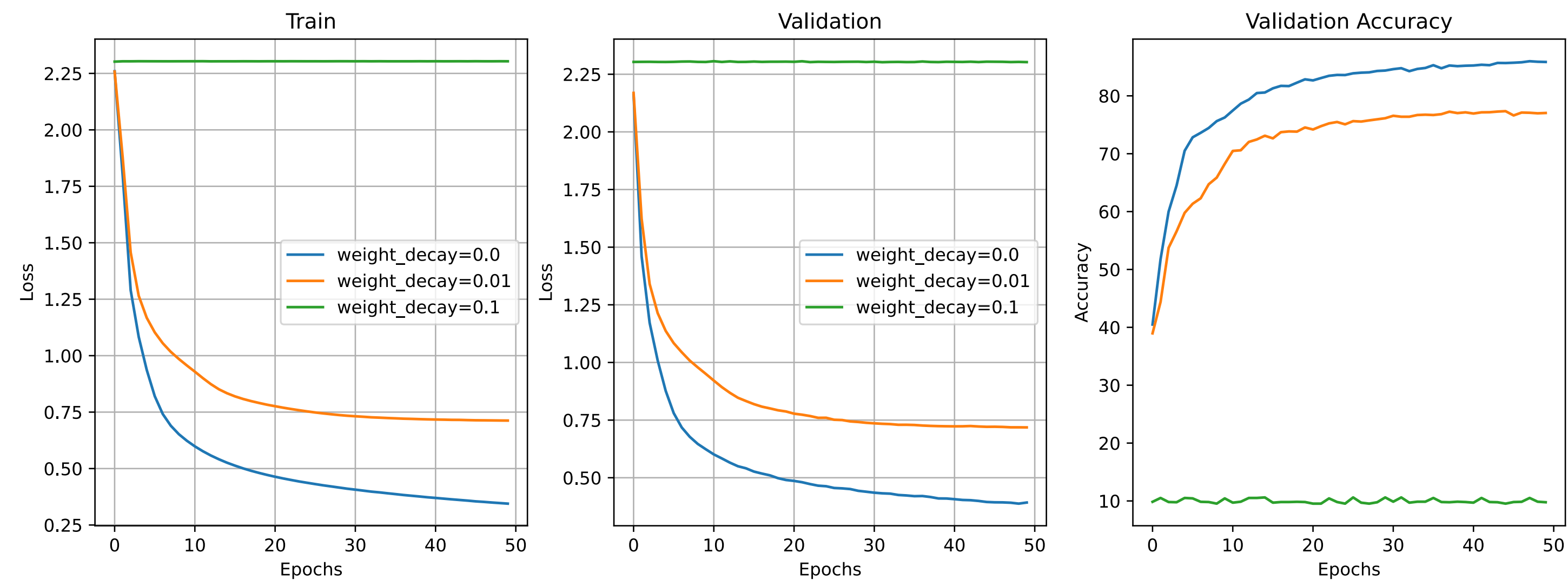
$$\begin{aligned}\theta_{t+1} &= \theta_t - \eta \frac{\partial \mathcal{L}}{\partial \theta} \\ &= \theta_t - \eta \left[\frac{\partial \ell}{\partial \theta} + \lambda \text{sign}(\theta) \right] \\ &= \theta_t - \eta \frac{\partial \ell}{\partial \theta} - \eta \lambda \text{sign}(\theta)\end{aligned}$$



- LASSO déplace les paramètres de la même façon quelle que soit la valeur (seul le signe compte)
- L'utilisation de norme L1 force la parcimonie (*sparsity*) ce qui peut être un avantage pour l'interprétabilité

Améliorer la généralisation

Exemple d'impact du weight-decay sur l'apprentissage



Dataset : FasionMNIST

Model : 3 layer FFN with sigmoid activations

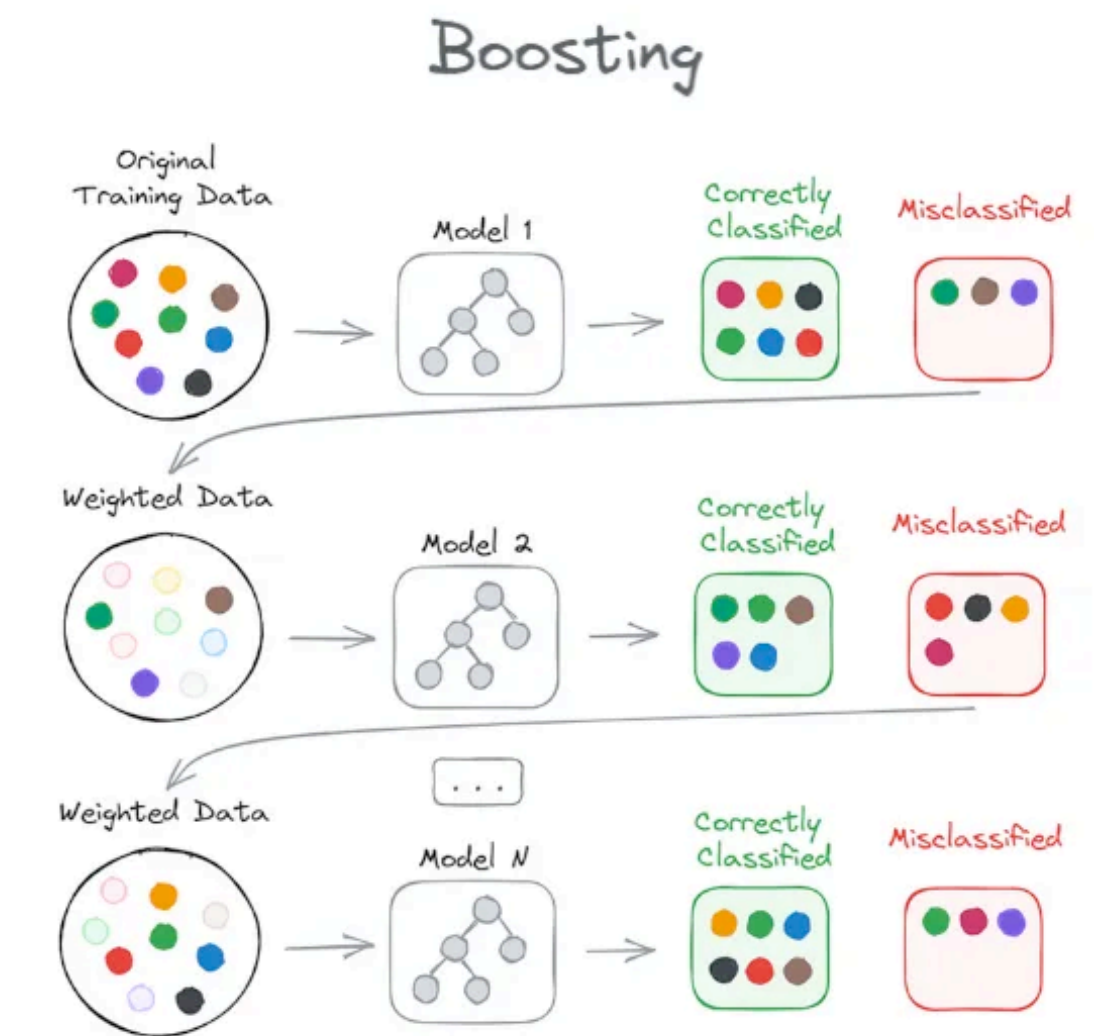
Learning rate : 0.01, epochs: 50, batch size: 32

10% data kept for validation

Améliorer la généralisation

Drop-out

- Vous vous rappelez du *bagging/boosting* ?
 - Apprendre plusieurs modèles pour une même tâche, sur des sous ensembles de données distincts
 - Combiner les sorties (vote majoritaire, fusion de scores)
- Fusion des scores : Mixture of classifiers
- Efficace mais coûteux (plusieurs modèles à entraîner)
- **Drop-out** exploite l'architecture des NN pour éviter d'apprendre plusieurs réseaux

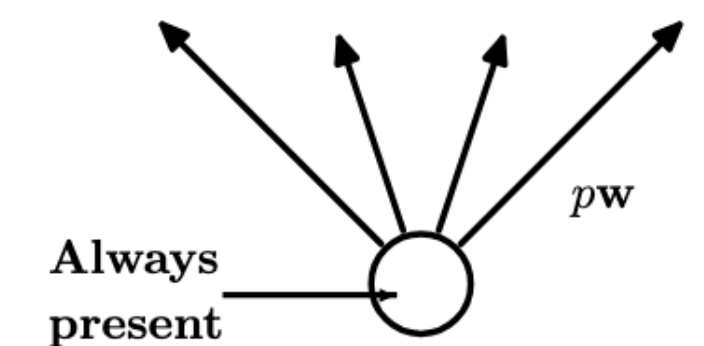
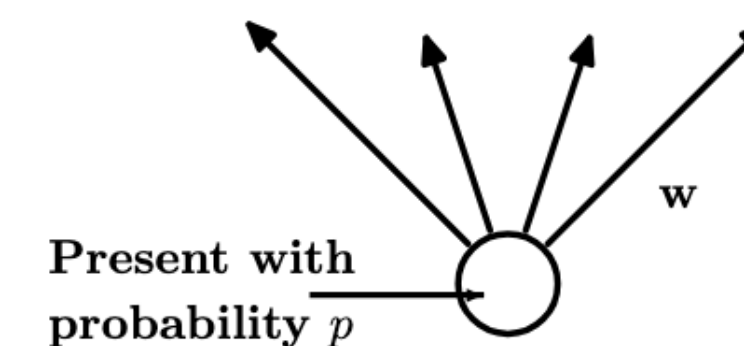
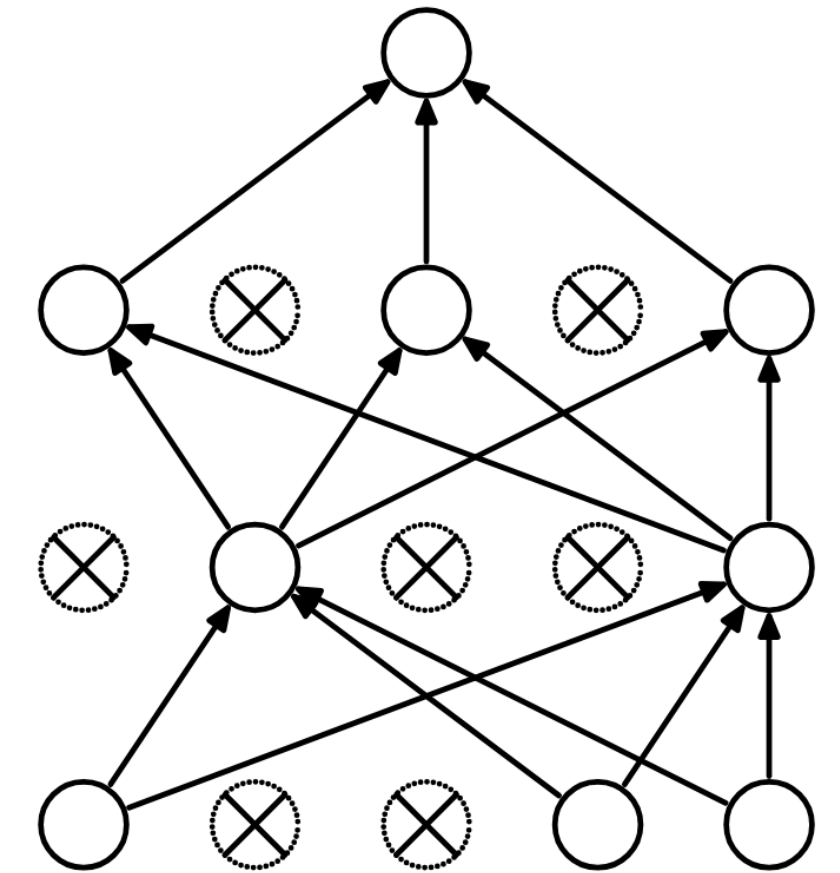
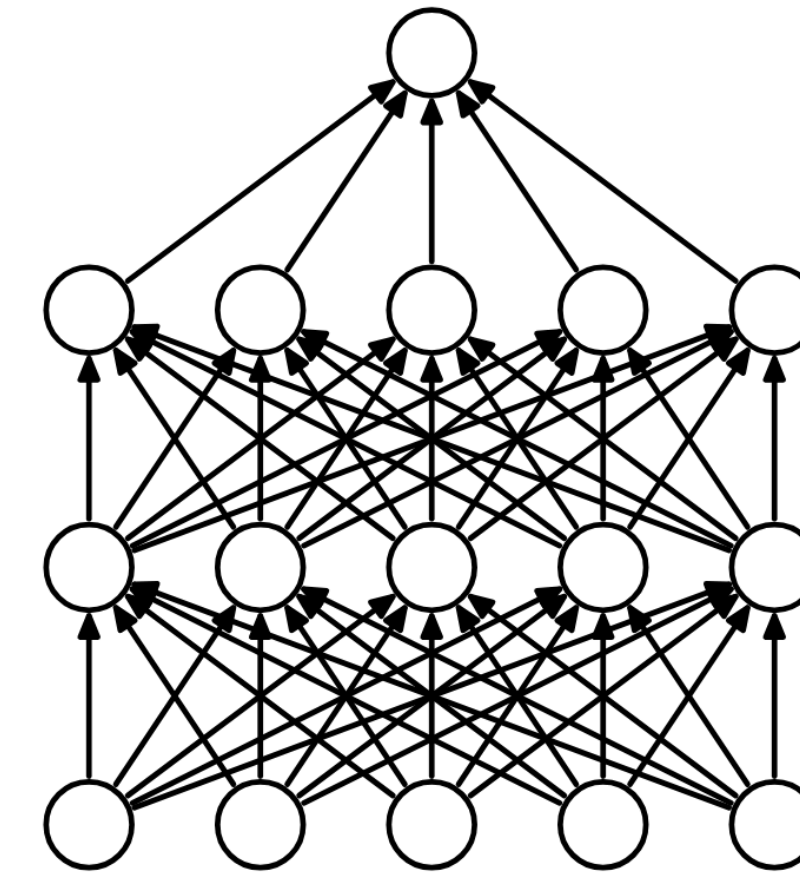


X →	Model A	→	0.3	0.2	0.5
X →	Model B	→	0.1	0.8	0.1
Moyenne arithmétique			0.2	0.5	0.3
Moyenne Géométrique			$\sqrt{0.03}$	$\sqrt{0.16}$	$\sqrt{0.05}$

Améliorer la généralisation

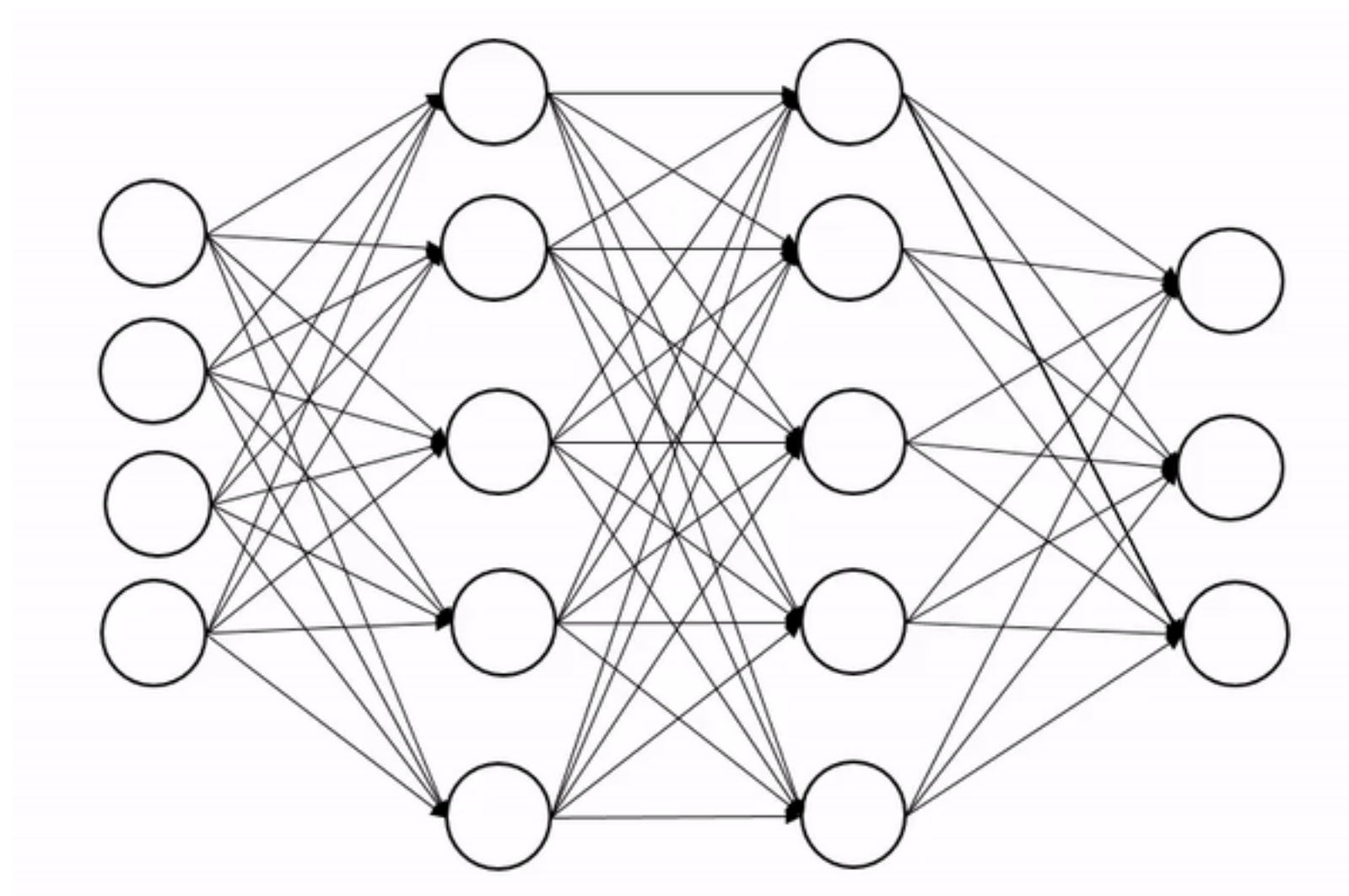
Drop-out

- Le drop-out consiste à désactiver des connections selon une probabilité p au cours de l'apprentissage
- Pour un batch, on désactive certains « neurones » $\Rightarrow$ on optimise un *sous-réseau* du réseau initial
- Si $\mathcal{N}$ est le nombre de neurones, on a $2^{\mathcal{N}}$ réseaux possibles
- **Et en inférence ?**
 - Option 1 : on tire un des sous réseaux aléatoirement (sous optimal)
 - Option 2 : on utilise tous les poids que l'on pondère par leur probabilité p
- **L'option 2 revient à prendre la moyenne géométrique des $2^{\mathcal{N}}$ sous modèles !**



Améliorer la généralisation

Drop-out : illustration

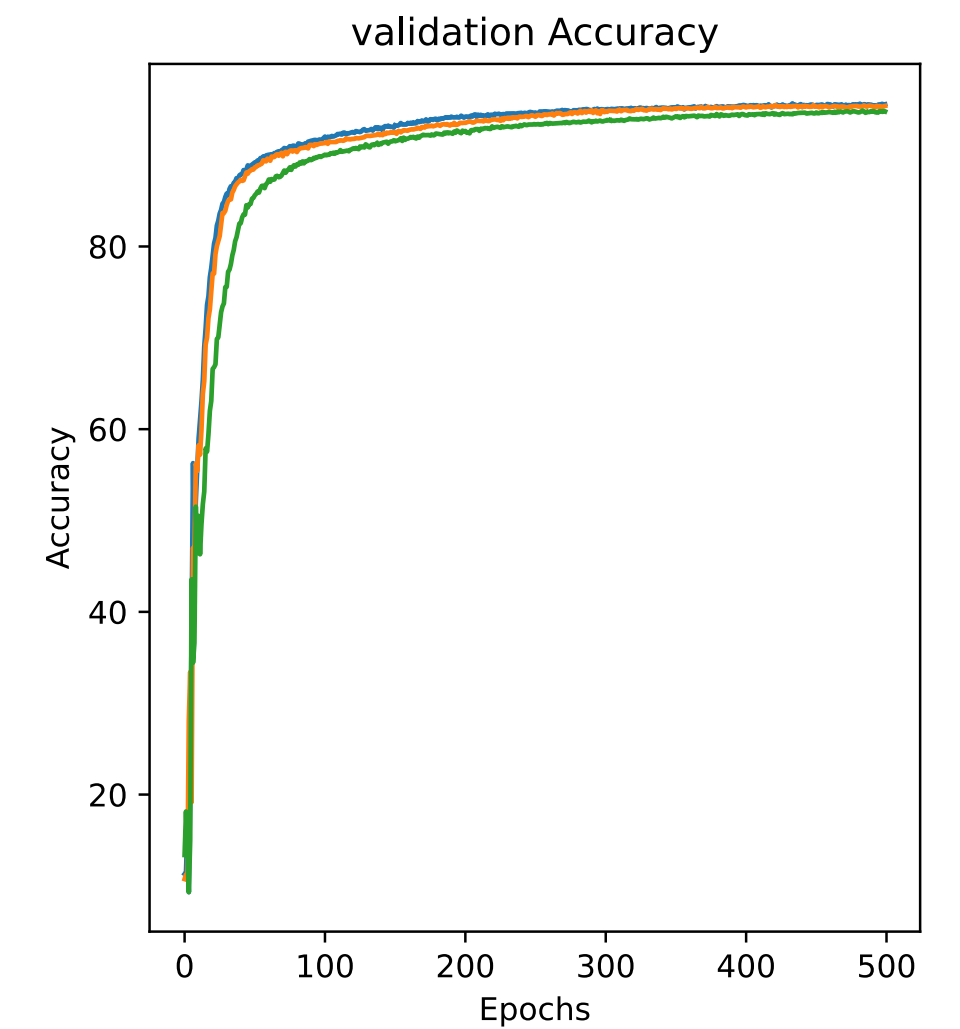
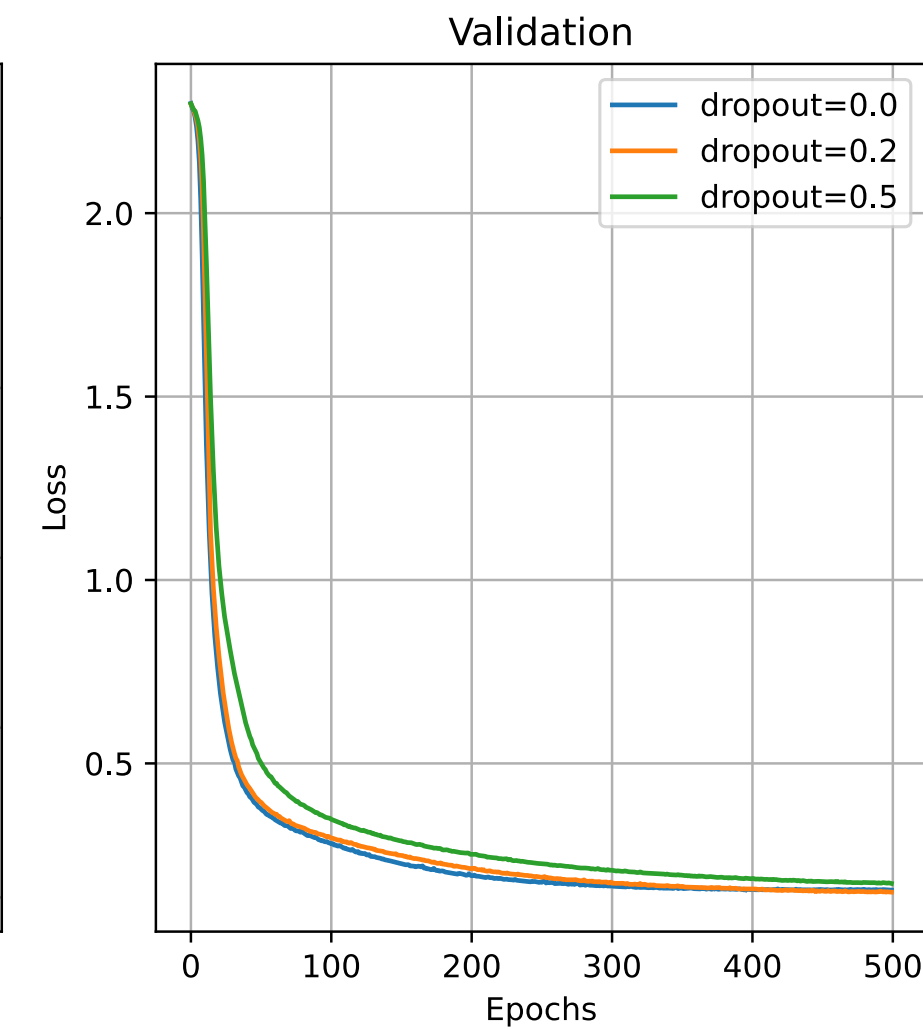
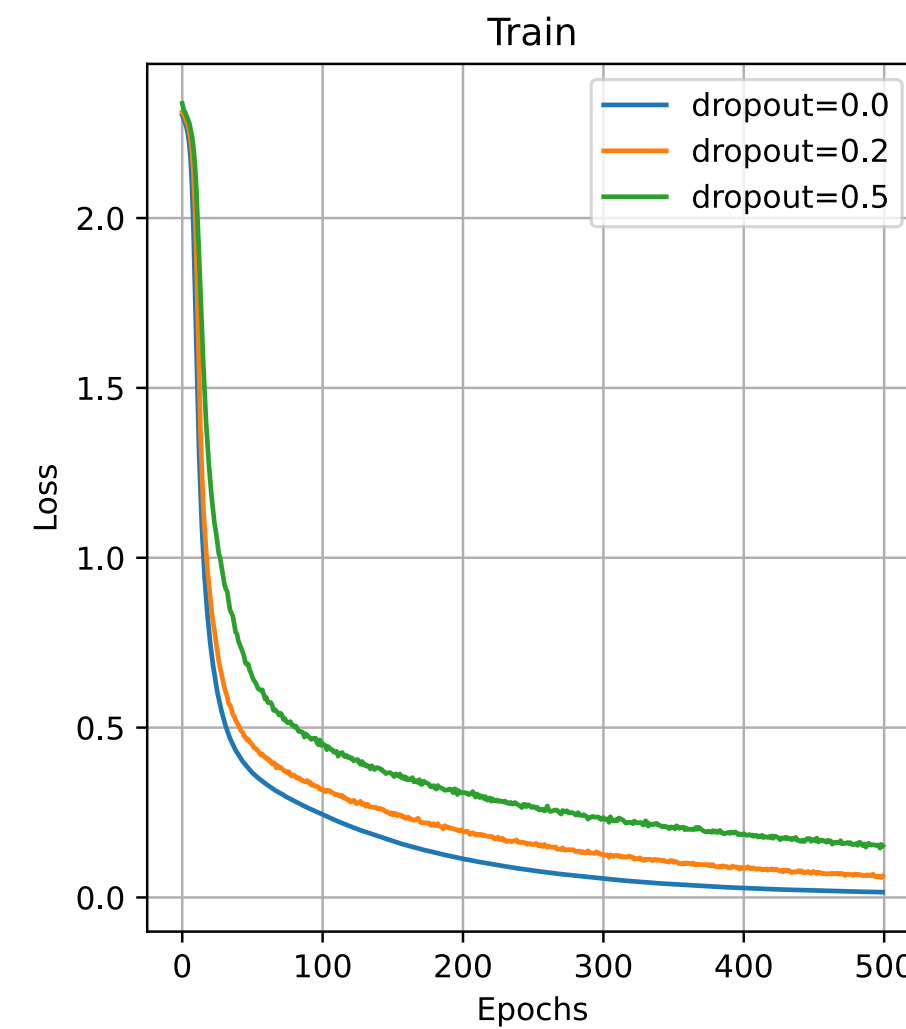


Dataset : FasionMNIST

Model : 3 layer FFN with sigmoid activations

Learning rate : 0.01, epochs: 50, batch size: 32

10% data kept for validation



Améliorer la généralisation

Augmentation de données

- Augmentation de données $\Rightarrow$ dupliquer les données d'apprentissage en leur apportant des modifications
- Par exemple :
 - Ajout de bruit dans un signal audio
 - Filtrage d'une image pour la rendre floue
 - Suppression de mots dans une phrase
- ⚠ l'augmentation de données ne crée pas de nouvelles données d'apprentissage !
- Elle altère les données existantes et en modifie la distribution
- Améliore la robustesse, la généralisation et peut être utile sur les petits jeux de données (limiter l'overfitting)

Améliorer la généralisation

Exemples d'augmentations de données

Audio

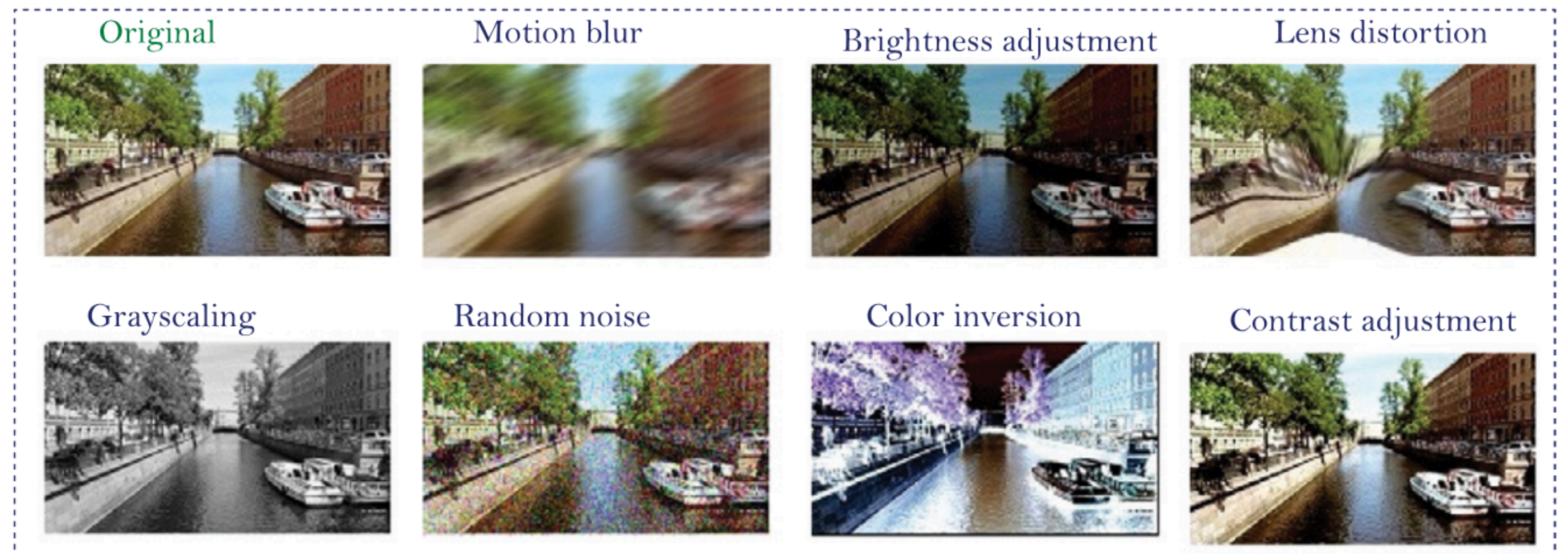
Tâche : segmentation de locuteurs

Image

Tâches : classification, segmentation

Augmentations :

- bruit blanc
- évènements sonores
- musique



Améliorer la généralisation

Exemples d'augmentations de données

	Top-1 accuracy (%)	Top-5 accuracy (%)
Baseline	48.13 ± 0.42	64.50 ± 0.65
Flipping	49.73 ± 1.13	67.36 ± 1.38
Rotating	50.80 ± 0.63	69.41 ± 0.48
Cropping	<i>61.95 ± 1.01</i>	<i>79.10 ± 0.80</i>
Color Jittering	49.57 ± 0.53	67.18 ± 0.42
Edge Enhancement	49.29 ± 1.16	66.49 ± 0.84
Fancy PCA	49.41 ± 0.84	67.54 ± 1.01

Their results find that the cropping geometric transformation results in the most accurate classifier

The italic value denote high performance according to the comparative metrics

Optimisation de l'apprentissage

Sommaire

4. Réutiliser l'existant

1. Principe du transfer learning
2. Finetuning
3. Foundation models

5. Améliorer la généralisation

1. Weight-decay
2. Drop-out
3. Augmentation de données

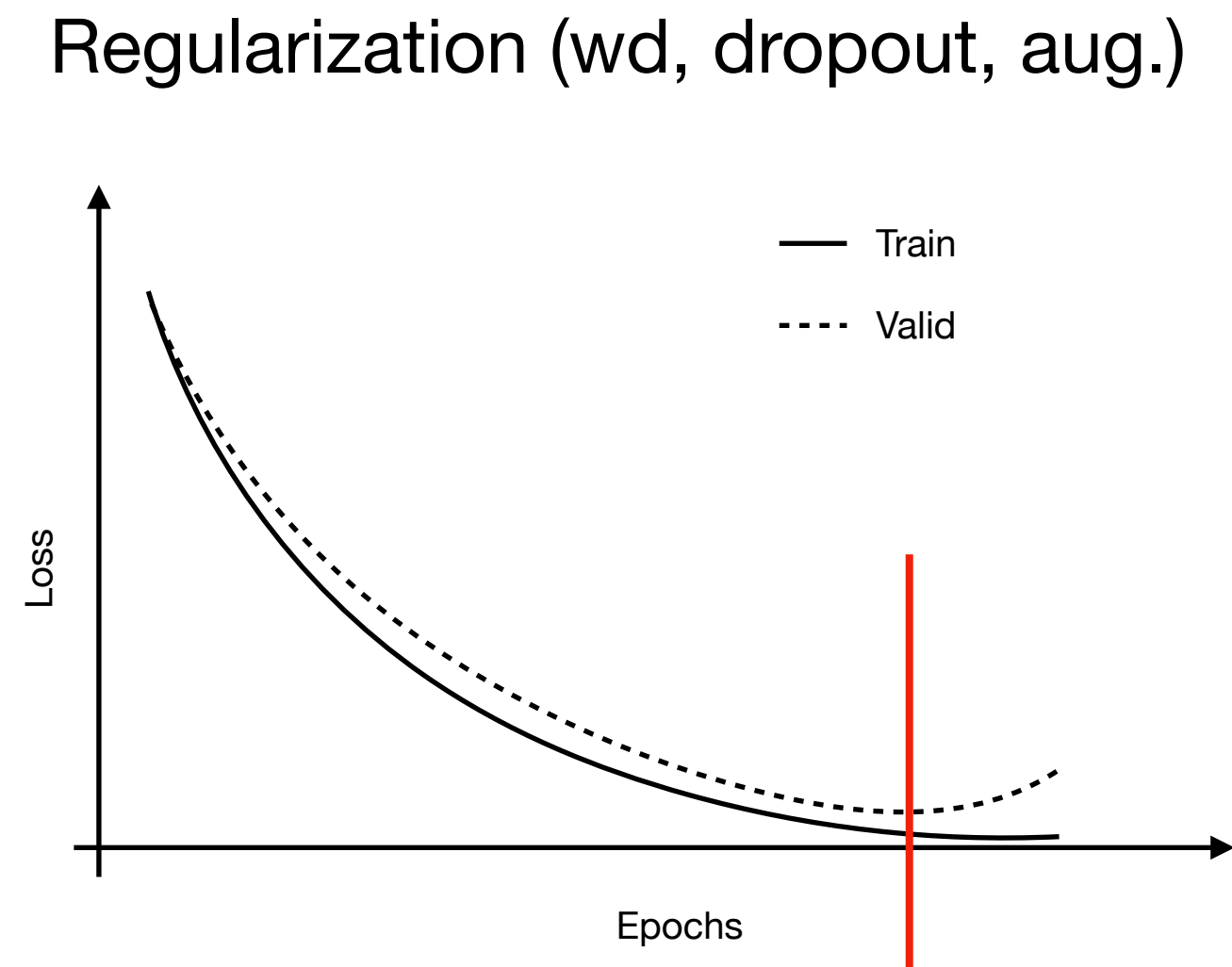
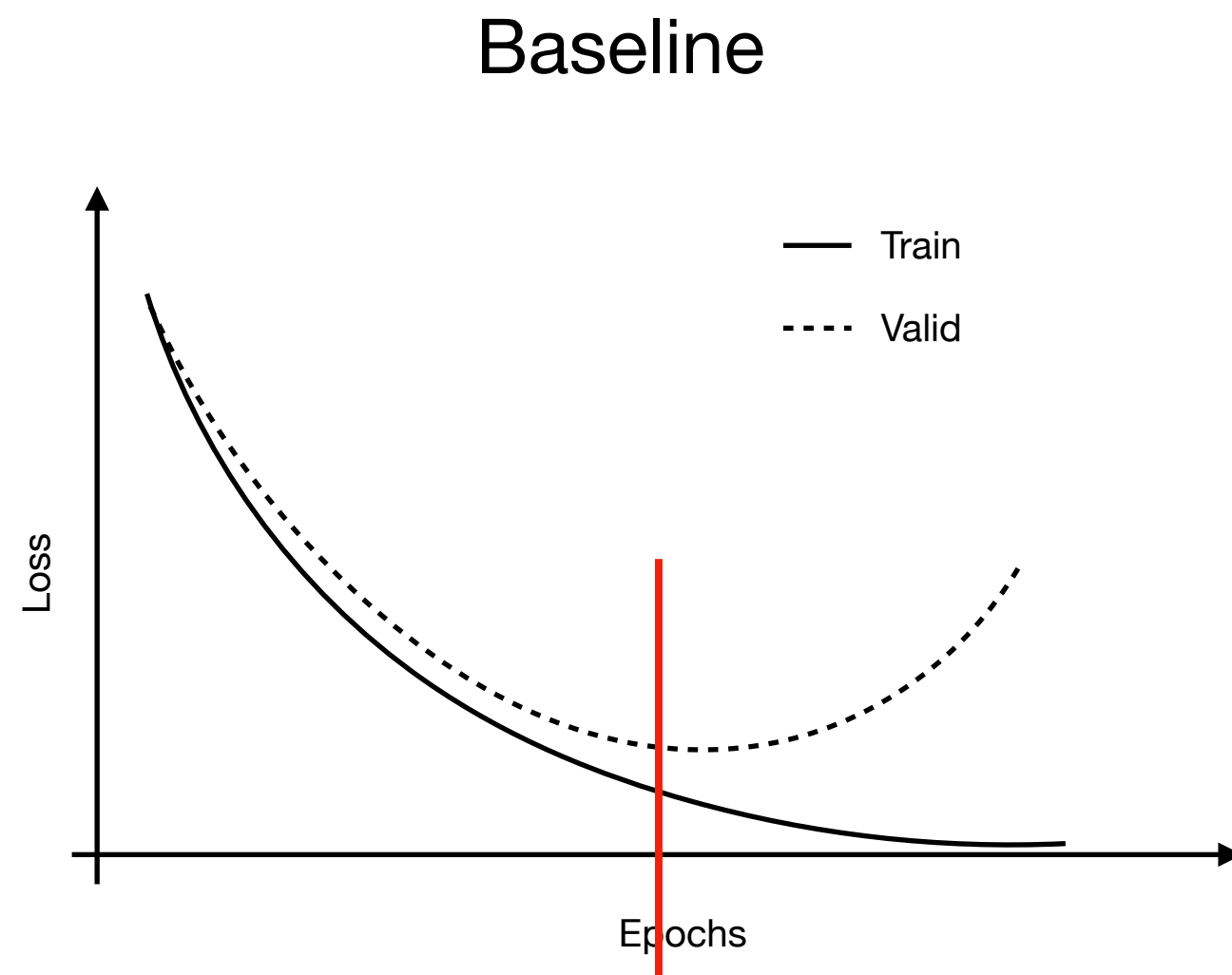
6. Fin de l'apprentissage

1. Early stopping
2. Model checkpointing

Arrêter l'apprentissage

Introduction

- **Régularisation** : permet de mieux généraliser et de garder un risque empirique proche du risque réel
- Comment utiliser le meilleur modèle après apprentissage ?

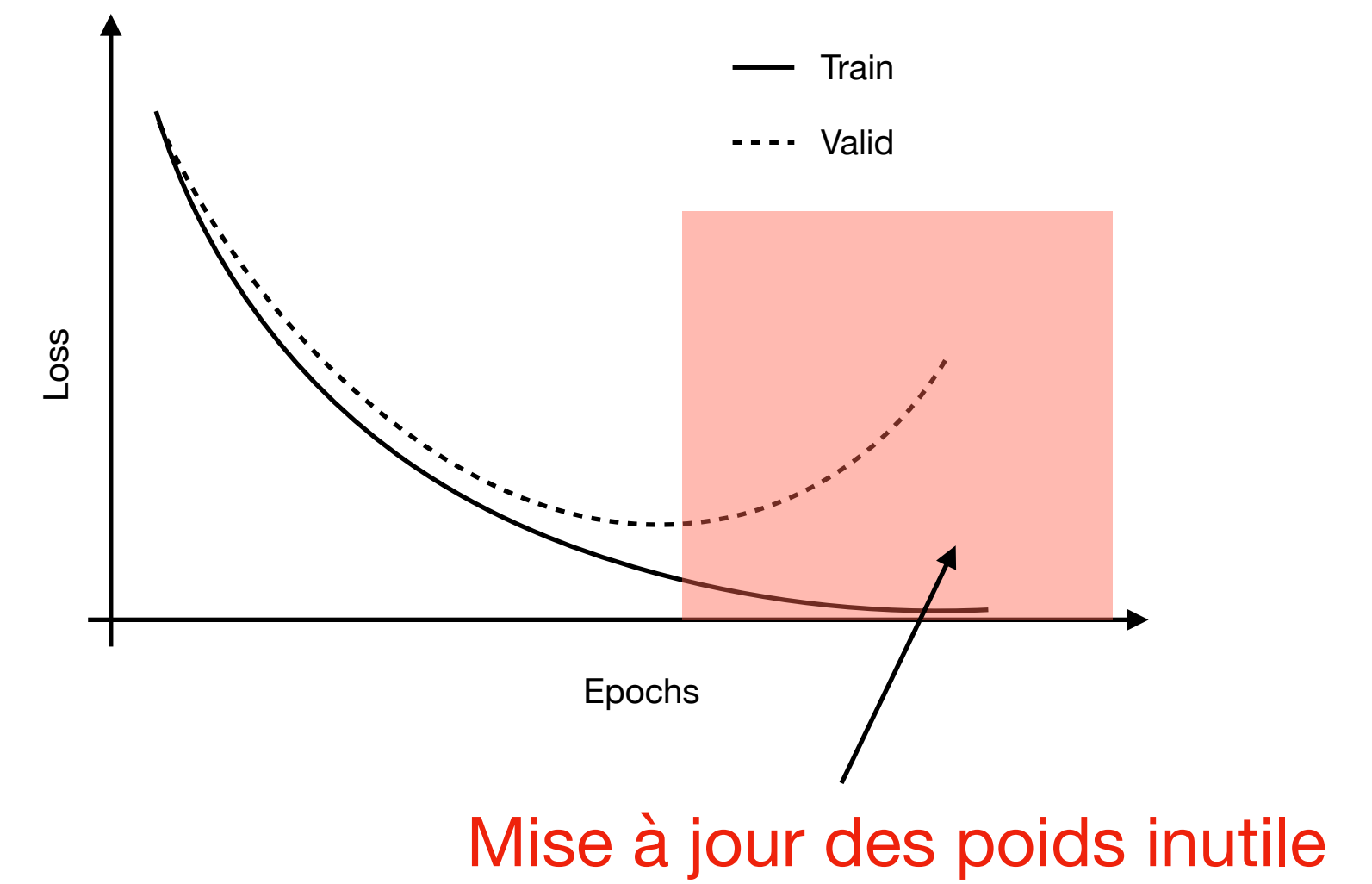


- Deux méthodes : *early stopping* et *checkpointing*

Arrêter l'apprentissage

Early stopping

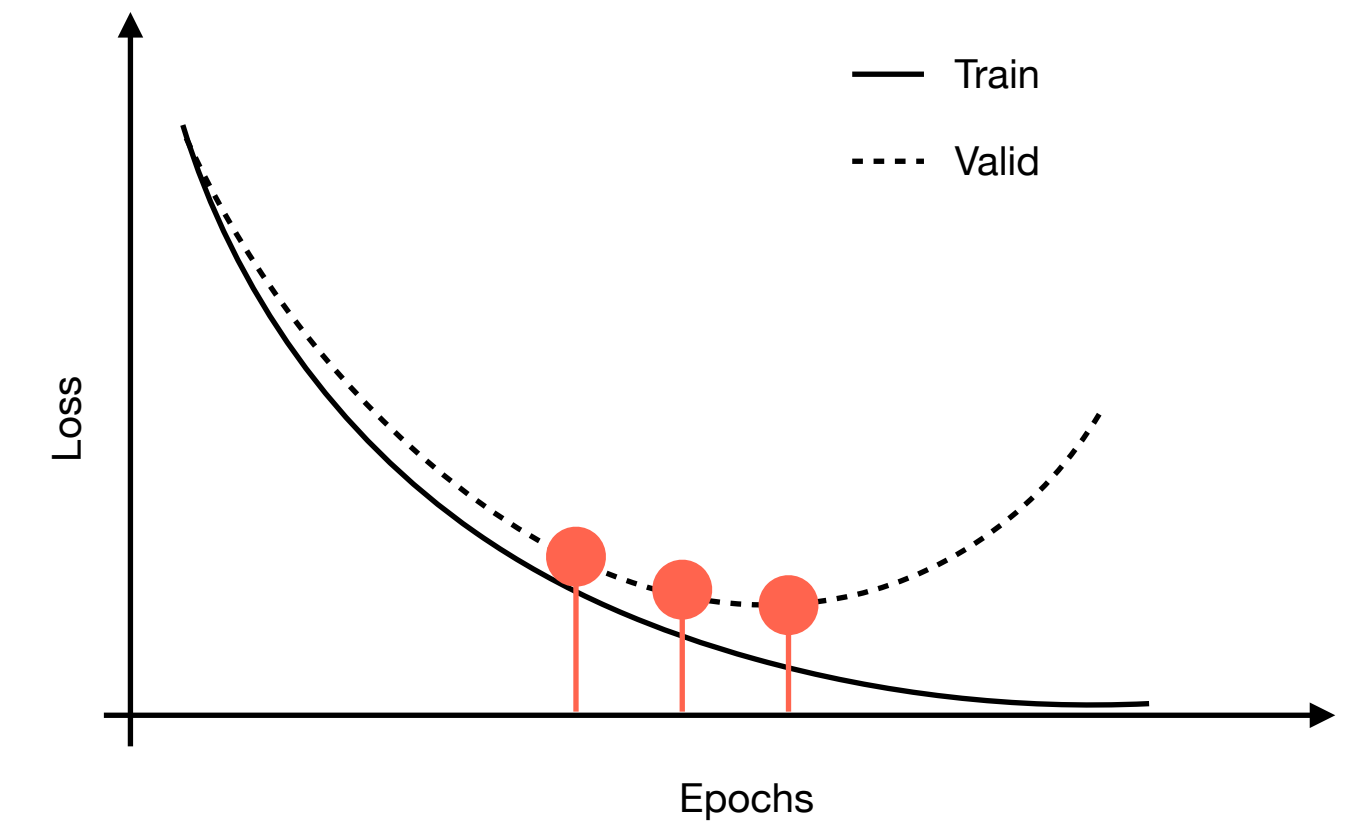
- Early stopping = s'arrêter tôt (smart)
- Il faut conserver le modèle qui a le meilleur score en validation !
- Une fois le minimum atteint sur la validation, on arrête l'apprentissage
- Lien théorique avec le **compromis biais-variance** : le point d'équilibre entre les deux donne le point d'arrêt
- En pratique :
 - Un hyper-paramètre : la patience = après combien d'epoch **sans amélioration de la loss de validation** je peux arrêter l'apprentissage
 - La loss de validation devient un **proxy** de la loss de généralisation



Arrêter l'apprentissage

Checkpointing

- **Checkpoint** = état d'un modèle à une étape de l'apprentissage
- Permet de conserver plusieurs versions du modèle à plusieurs étapes de l'optimisation
- Pratique courante : on conserve les n meilleurs modèles
- On trouve parfois les scores des systèmes sous la forme 1-best et 5-best
- Ils permettent également de reprendre l'apprentissage en cas d'arrêt prématuré
 - Attention à conserver l'état de l'optimiseur et du scheduler !!



	Top-1 accuracy (%)	Top-5 accuracy (%)
Baseline	48.13 ± 0.42	64.50 ± 0.65
Flipping	49.73 ± 1.13	67.36 ± 1.38
Rotating	50.80 ± 0.63	69.41 ± 0.48
Cropping	<i>61.95 ± 1.01</i>	<i>79.10 ± 0.80</i>
Color Jittering	49.57 ± 0.53	67.18 ± 0.42
Edge Enhancement	49.29 ± 1.16	66.49 ± 0.84
Fancy PCA	49.41 ± 0.84	67.54 ± 1.01

Their results find that the cropping geometric transformation results in the most accurate classifier

The italic value denote high performance according to the comparative metrics

Accélérer l'apprentissage

Du point de vue de l'implémentation

- Globalement, les réseaux de neurones artificiels profonds sont longs à optimiser
- Plusieurs raisons à cela:
 - La taille des modèles : les étapes de *forward/backward* sont coûteuses par la taille des modèles
 - La taille des jeux de données : faire une *epoch* peut nécessiter de nombreuses minutes si le jeu de données est grand
 - La difficulté de convergence : certaines tâches sont plus difficile que d'autres, et il faut parfois de nombreuses epochs pour obtenir un modèle performant
- Les éléments vus dans le cours permettent en partie d'accélérer l'apprentissage (convergence plus rapide notamment)
- Il existe des astuces du point de vue infrastructure de calcul pour aller plus vite aussi

Accélérer l'apprentissage

Précision mixte

- Par défaut, PyTorch travaille avec des valeurs flottantes en 32 bits (type `float` de torch)
- Cette précision est nécessaire pour de nombreuses opérations (moyenne de la loss par exemple)
- Cependant, certaines opérations sont bien plus efficaces avec une précision plus faible (16 bits par exemple)
- La précision mixte consiste à modifier le type des données en fonction des éléments de la boucle d'apprentissage et du réseau
- Par exemple, les couches de convolution et linéaires seront en `float16` et le calcul de la loss en `float32`
- **Avantages :**
 - Accélération drastique des opérations (multiplications, additions) (2-3x)
 - Gain en mémoire et en transfert de données vers le GPU -> apprentissage plus rapide
- La précision mixte est automatisée dans PyTorch

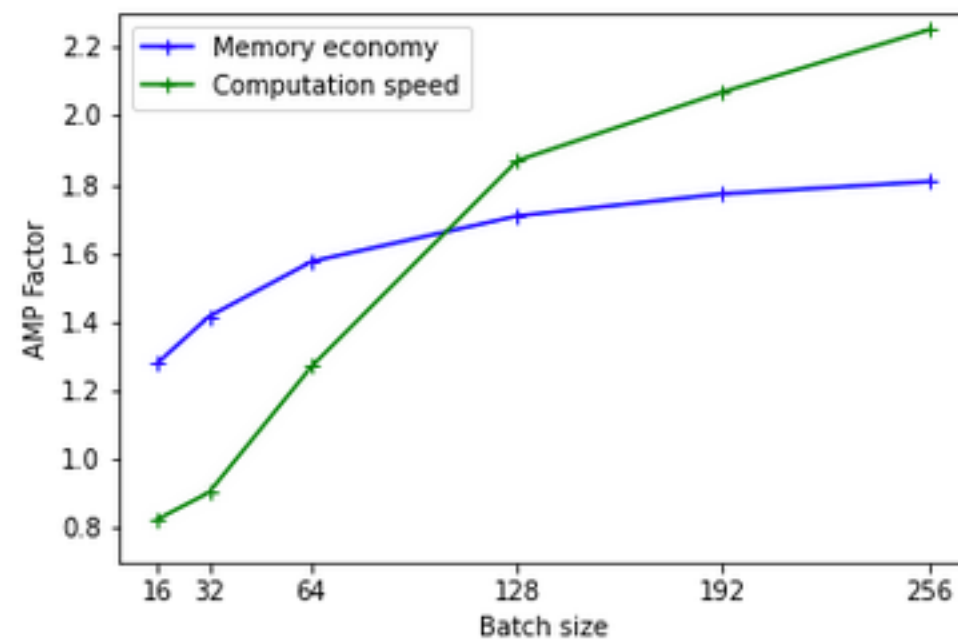
Accélérer l'apprentissage

Précision mixte : exemple d'implémentation

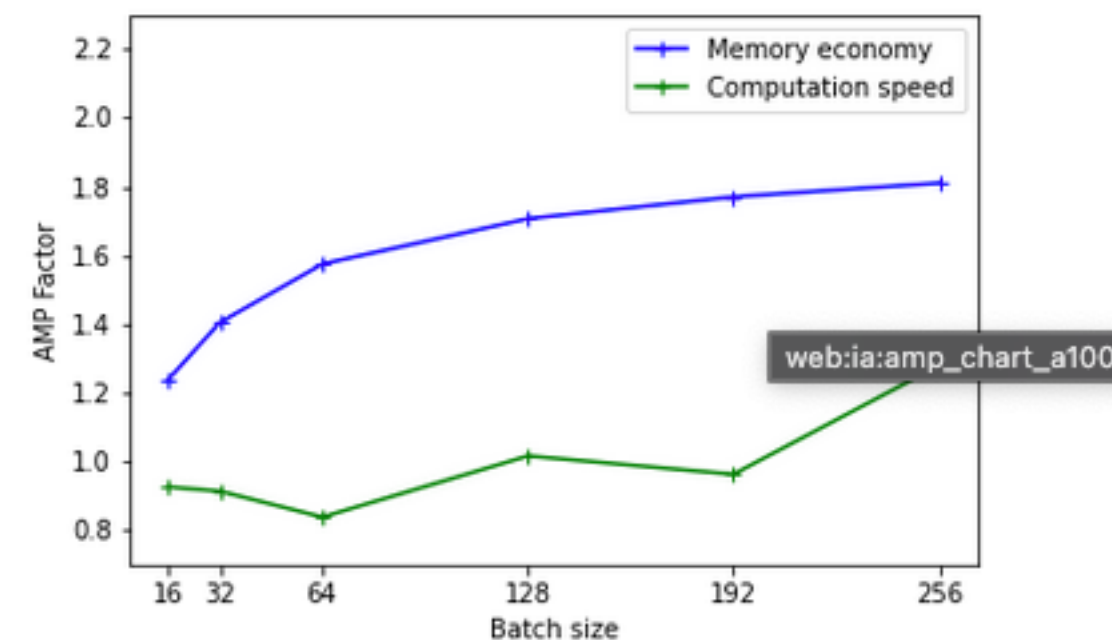
Sans précision mixte (boucle classique)

```
net = make_model(in_size, out_size, num_layers)
opt = torch.optim.SGD(net.parameters(), lr=0.001)

start_timer()
for epoch in range(epochs):
    for input, target in zip(data, targets):
        output = net(input)
        loss = loss_fn(output, target)
        loss.backward()
        opt.step()
        opt.zero_grad() # set_to_none=True here can modestly improve performance
end_timer_and_print("Default precision:")
```



Jean Zay V100



Jean Zay A100

Intégration de la précision mixte

```
use_amp = True

net = make_model(in_size, out_size, num_layers)
opt = torch.optim.SGD(net.parameters(), lr=0.001)
scaler = torch.amp.GradScaler("cuda", enabled=use_amp)

start_timer()
for epoch in range(epochs):
    for input, target in zip(data, targets):
        with torch.autocast(device_type=device, dtype=torch.float16, enabled=use_amp):
            output = net(input)
            loss = loss_fn(output, target)
        scaler.scale(loss).backward()
        scaler.step(opt)
        scaler.update()
        opt.zero_grad() # set_to_none=True here can modestly improve performance
end_timer_and_print("Mixed precision:")
```

- Intégration très facile dans un code déjà existant
- **Attention** : uniquement valable sur GPU Cuda
- **Attention** : il faut bien sauvegarder l'état du scaler lors du checkpointing (comme l'optimizer)

Accélérer l'apprentissage

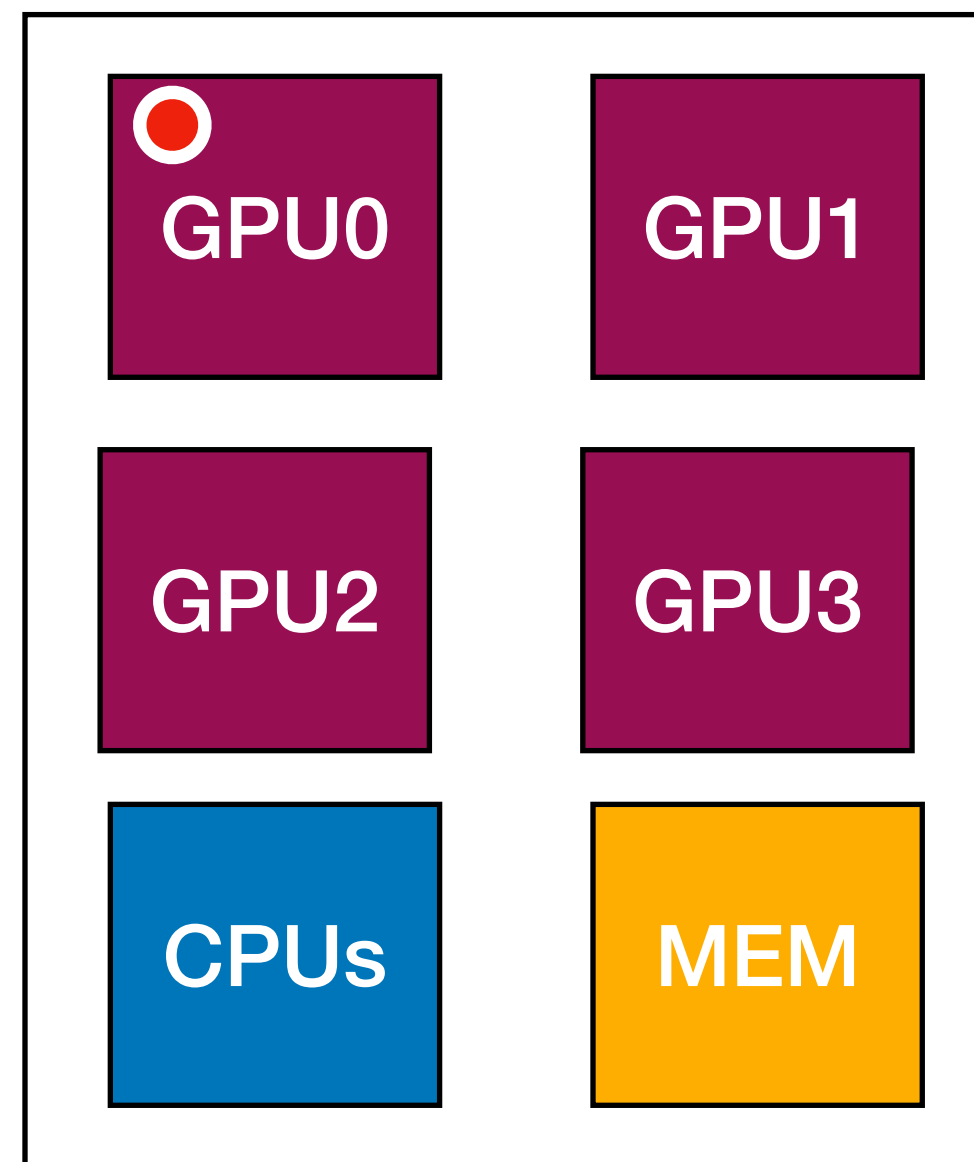
Infrastructure de calcul : multi-GPU, multi-Nœud

- Les réseaux de neurones ne sont qu'une série de produits matriciels
- Les GPU (Graphical Processing Unit) sont donc particulièrement adaptés pour accélérer l'apprentissage !
- Par défaut, PyTorch utilise un seul GPU
 - **Important :** même si vous êtes connecté sur un nœud de calcul avec plusieurs cartes GPU, PyTorch n'en utilisera qu'une !
- Dans le cas de l'apprentissage de grands modèles avec de large jeux de données, il est intéressant de paralléliser l'apprentissage
- Deux options s'offrent à nous:
 - Mono-nœud et multi-GPU
 - Multi-nœud et multi-GPU

Accélérer l'apprentissage

Mono-GPU

Node 01



- Je suis connecté au Node 01 et j'ai demandé les 4 GPU disponibles
- Le code à côté va utiliser uniquement un seul GPU (probablement GPU0)
- **Pas de parallélisation !**

```
import torch
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from datautils import MyTrainDataset
```

```
class Trainer:
    def __init__(
        self,
        model: torch.nn.Module,
        train_data: DataLoader,
        optimizer: torch.optim.Optimizer,
        gpu_id: int,
        save_every: int,
    ) -> None:
        self.gpu_id = gpu_id
        self.model = model.to(gpu_id)
        self.train_data = train_data
        self.optimizer = optimizer
        self.save_every = save_every

    def _run_batch(self, source, targets):
        self.optimizer.zero_grad()
        output = self.model(source)
        loss = F.cross_entropy(output, targets)
        loss.backward()
        self.optimizer.step()

    def _run_epoch(self, epoch):
        b_sz = len(next(iter(self.train_data))[0])
        print(f"[GPU{self.gpu_id}] Epoch {epoch} | Batchsize: {b_sz} | Steps: {len(self.train_data)}")
        for source, targets in self.train_data:
            source = source.to(self.gpu_id)
            targets = targets.to(self.gpu_id)
            self._run_batch(source, targets)

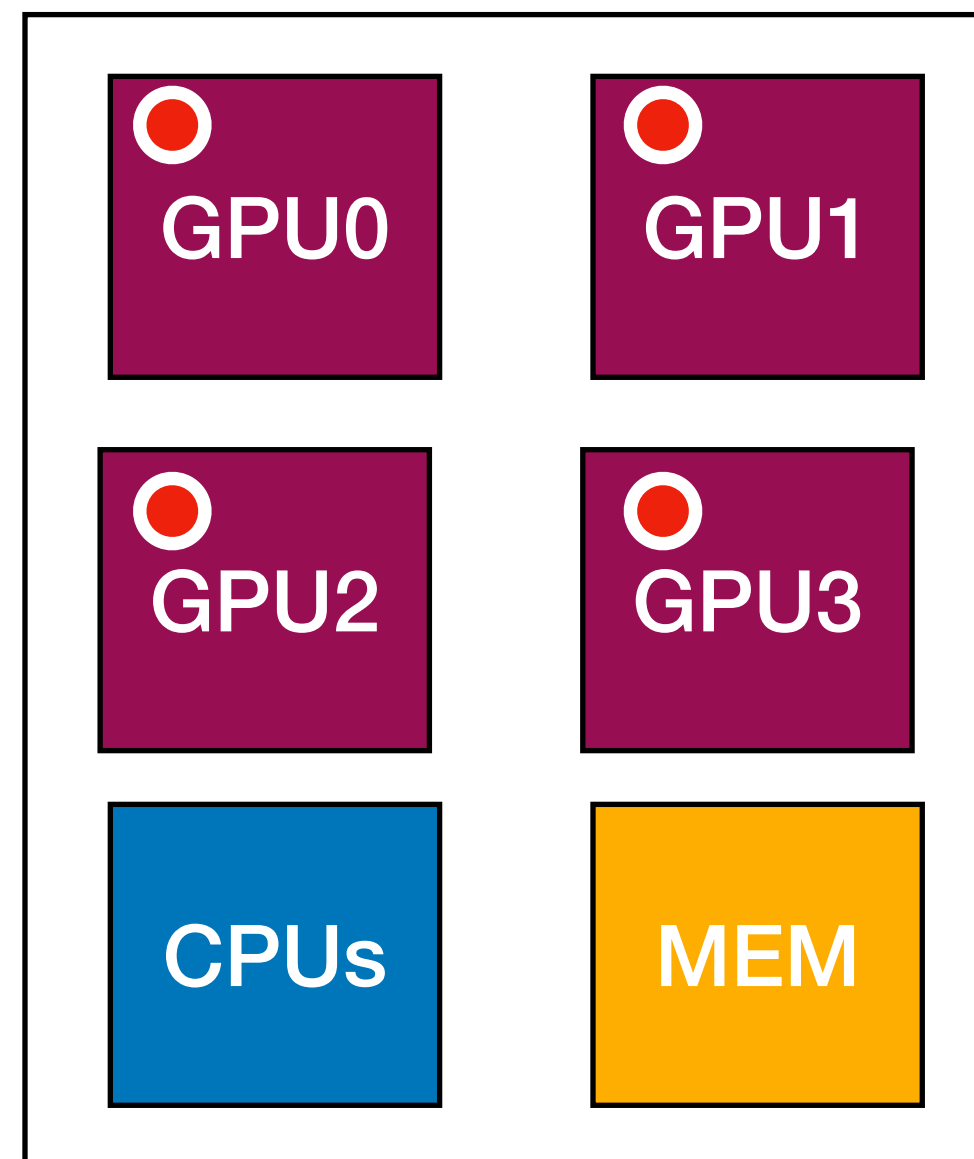
    def train(self, max_epochs: int):
        for epoch in range(max_epochs):
            self._run_epoch(epoch)
            if epoch % self.save_every == 0:
                self._save_checkpoint(epoch)

def main(device, total_epochs, save_every, batch_size):
    dataset, model, optimizer = load_train_objs()
    train_data = prepare_dataloader(dataset, batch_size)
    trainer = Trainer(model, train_data, optimizer, device, save_every)
    trainer.train(total_epochs)
```


Accélérer l'apprentissage

Multi-GPU // Mono-Nœud

Node 01



- Torch.multiprocessing et DistributedDataParallel permettent de gérer l'apprentissage sur plusieurs GPU au sein d'un seul nœud
- Le code précédent demande quelques ajustements mais l'implémentation reste très simple
- **Ici, les batch sont répartis sur tous les GPUs disponibles**
 - Si on a 2000 batches dans notre dataset, chaque GPU s'occupera de l'apprentissage sur 500 batches
 - Le modèle est recopié sur chaque GPU et la mise à jour des poids est synchronisée par DDP et torch.mp

```
import torch.multiprocessing as mp
from torch.utils.data.distributed import DistributedSampler
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.distributed import init_process_group, destroy_process_group

# setup function for DDP
def ddp_setup(rank: int, world_size: int):
    """
    Args:
        rank: Unique identifier of each process
        world_size: Total number of processes
    """
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "12355"
    torch.cuda.set_device(rank)
    init_process_group(backend="nccl", rank=rank, world_size=world_size)

# Model wrapper for DDP
self.model = DDP(model, device_ids=[gpu_id])

# Change the data loader
train_data = torch.utils.data.DataLoader(
    dataset=train_dataset,
    batch_size=32,
    shuffle=False, # We don't shuffle
    sampler=DistributedSampler(train_dataset), # Use the Distributed Sampler here.
)

def _run_epoch(self, epoch):
    b_sz = len(next(iter(self.train_data))[0])
    self.train_data.sampler.set_epoch(epoch) # call this additional line at every epoch
    for source, targets in self.train_data:
        ...
        self._run_batch(source, targets)

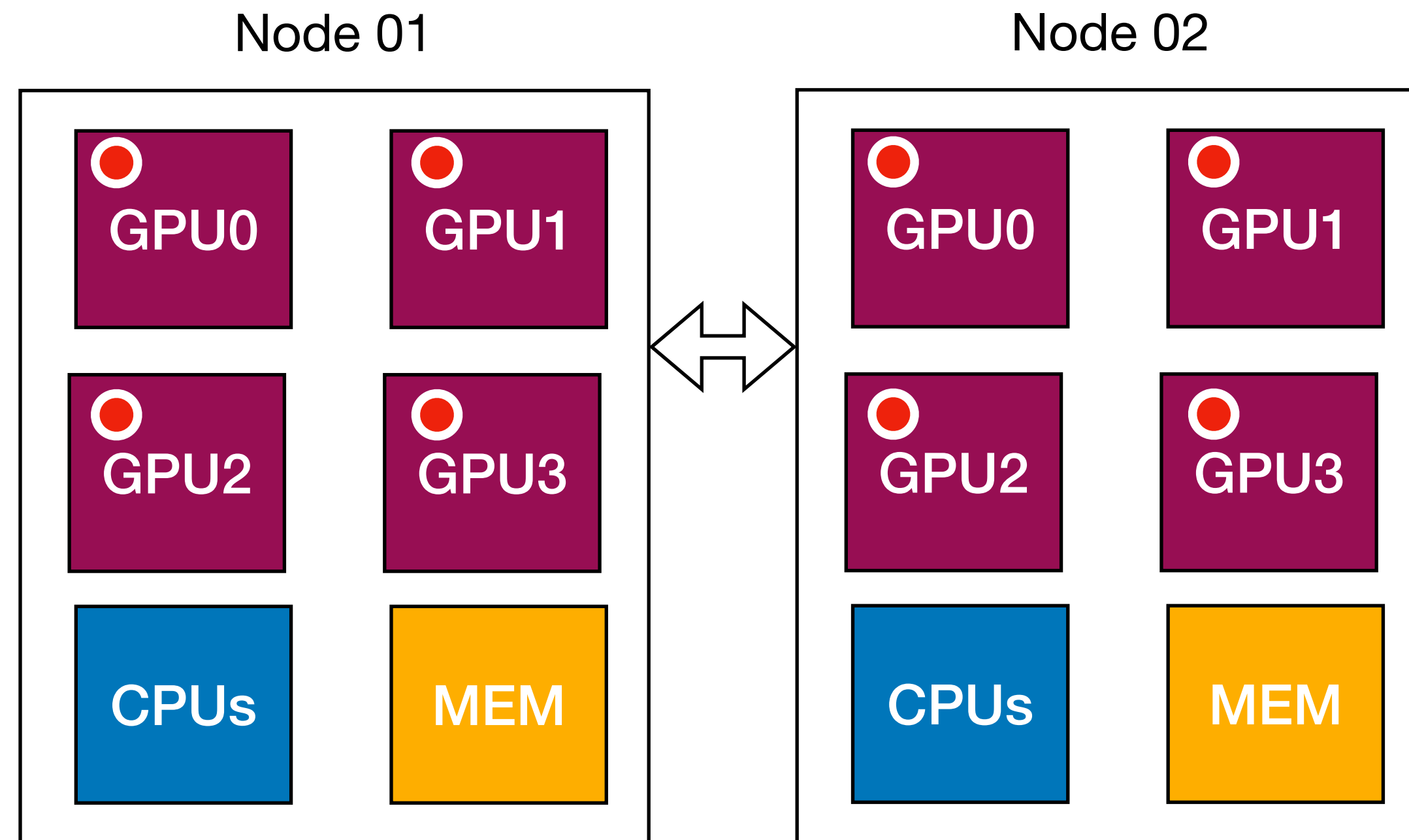
# in the training loop
- ckp = self.model.state_dict()
+ ckp = self.model.module.state_dict()
...
- if epoch % self.save_every == 0:
+ if self.gpu_id == 0 and epoch % self.save_every == 0:
    self._save_checkpoint(epoch)

- def main(device, total_epochs, save_every):
+ def main(rank, world_size, total_epochs, save_every):
+     ddp_setup(rank, world_size)
    dataset, model, optimizer = load_train_objs()
    train_data = prepare_dataloader(dataset, batch_size=32)
-     trainer = Trainer(model, train_data, optimizer, device, save_every)
+     trainer = Trainer(model, train_data, optimizer, rank, save_every)
    trainer.train(total_epochs)
+     destroy_process_group()

if __name__ == "__main__":
    import sys
    total_epochs = int(sys.argv[1])
    save_every = int(sys.argv[2])
-     device = 0 # shorthand for cuda:0
-     main(device, total_epochs, save_every)
+     world_size = torch.cuda.device_count()
+     mp.spawn(main, args=(world_size, total_epochs, save_every,), nprocs=world_size)
```

Accélérer l'apprentissage

Multi-GPU // Multi-Nœud



- Pour l'apprentissage multi-nœud, il faut permettre aux différents nœuds (machines) de communiquer entre elles et de se synchroniser
- Heureusement, il existe **torchrun** qui permet de gérer cette communication
- En mode multi-GPU, mono-nœud, **torchrun** est plus robuste que DDP car il gère les exceptions
 - ➡ Plus besoins de passer les **world_size** et **rank** au trainer
- En mode Multi-GPU, multi-nœud, le code est très peu modifié
- Très pratique avec SLURM !
- **Attention** : la communication entre machines est souvent lente et peut ralentir l'apprentissage. Si possible, il vaut mieux rester sur une seule machine avec plusieurs GPU que plusieurs machine avec 1 GPU

Je vous laisse consulter cette ref pour les détails: https://docs.pytorch.org/tutorials/beginner/ddp_series_fault_tolerance.html

Conclusions

1. Algorithmes d'optimisation

- Le **learning rate** est le paramètre le plus important pour l'optimisation du réseau
- Il est possible de mettre à jour le learning rate au cours de l'apprentissage à l'aide de **schedulers**
- Plusieurs algorithmes permettent de mettre à jour les poids (**optimiseurs**) : SGD, AdaGrad, RMSProp, Adam, AdamW...

2. Modification de l'espace de recherche

- L'architecture du réseau a un impact sur l'espace des paramètres
- Les **connexions résiduelles** permettent de **simplifier cet espace** tout en permettant de créer des réseaux profonds
- La **largeur des couches** permet aussi de simplifier l'espace des paramètres (mais augmente le coût de calcul)

3. Stabilisation de l'apprentissage

- Le choix de la taille de batch influe sur la **stabilité de l'apprentissage**
- Il faut choisir la plus grande taille possible **sans dégrader** la généralisation
- Il est possible d'accumuler les gradients en cas de mémoire limitée, et de contraindre la norme du gradient pour éviter les valeurs trop grandes

Conclusions

4. Réutiliser l'existant

- Attention à **toujours faire un état de l'art** avant de se lancer dans un projet de machine learning
- Le ***transfer learning*** permet de réutiliser des modèles préalablement entraînés sur des tâches proches
- Les **modèles de fondation** (*foundation models*) permettent d'obtenir des représentations optimisées des données pour diverses tâches et sont très performants

5. Améliorer la généralisation

- La régularisation consiste à **pénaliser l'apprentissage** pour éviter l'*overfitting*
- Le ***weight-decay*** réduit la valeur des paramètres en ajoutant une pénalité à la fonction de coût
- Le ***drop-out*** est une extension du *boosting* aux DNN et consiste à désactiver des neurones pendant l'apprentissage (on apprend des sous-modèles)
- L'**augmentation de données** permet de modifier la distribution des données pour améliorer la généralisation

6. Fin de l'apprentissage

1. L'***early stopping*** permet d'arrêter l'apprentissage lorsque la loss de validation est minimale (meilleur modèle apparent)
2. Le ***checkpointing*** consiste à sauvegarder des modèles à différentes étapes de l'apprentissage